A COURSE FOR NEW LEARNERS · 150 SLIDES

How the ESP32 drone works

The parts, the firmware design, and the code — line by line, in plain words.

XW_Z1 V2.0 board

ESP32 + MPU6500

Esp32_drone.ino

Android · iOS apps

WELCOME

What you will be able to do after this course

Explain how it flies

Why four spinning props can climb, tilt and turn — and why it falls without a computer.

Name every part

The ESP32, the motion sensor, the motor drivers, the battery sense — and what each pin does.

Read the firmware

Follow one loop of the code from sensor reading to motor output, 250 times a second.

Change it safely

Know which safety layers exist, run the tests, and make a small change without breaking them.

WELCOME

The drone at a glance

4

brushed 8520 motors, driven straight from the battery through MOSFETs

1

motion sensor (MPU6500). No barometer, no GPS, no compass

250

times per second the control loop reads, decides and sets the motors

One ESP32 chip does everything: the flight control, Wi-Fi and Bluetooth to the phone, and the SBUS radio receiver input.

WELCOME

Course map: 12 sections

#	Section	#	Section
1	Welcome	7	Mixer and motors
2	How a quadcopter flies	8	Radio inputs: SBUS and CRTP
3	The hardware parts	9	Flight modes and safety
4	Firmware architecture	10	Navigation
5	Reading the motion sensor	11	Testing and workflow
6	PID control	12	Wrap-up

Sections 2–3 need no code. From section 4 on, each idea is shown in the real firmware.

WELCOME

How to read the code slides

```
// a comment explains the line below
#define LOOP_TIME_US  4000

void loop() {
  mpuReadRaw();  // read sensor
}
```

Grey = a comment. The compiler ignores it; it is for humans.

Orange = a C++ keyword such as *void*, *if*, *float*.

White = the code itself. Names come from the real file.

Code is shortened with “…” when a part is not needed for the idea. The footer says where it lives.

WELCOME

Before you touch the drone: three rules

1. **Props off** for every test on the desk. A spinning prop cuts skin.

2. **Know the stop.** CUT MOTORS in the app, or the RC disarm, before you arm.

3. **Change one thing at a time,** then test it. Two changes at once hide which one broke it.

02

How a quadcopter flies

Four props, three axes, and why it cannot fly without a computer.

HOW IT FLIES

A quadcopter is four fans pushing air down

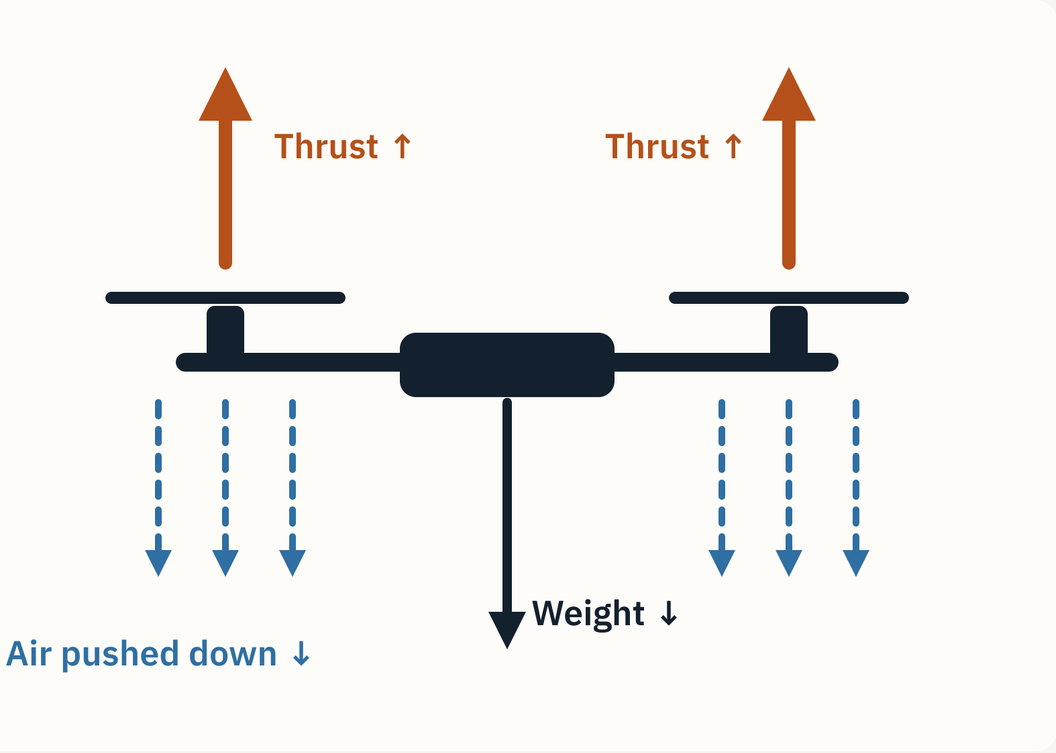
Each prop throws air down. The air pushes the prop up with the same force: thrust.

Faster prop → more air per second → more thrust on that corner.

No moving surfaces: unlike a plane, every move comes from changing four motor speeds.

EXAMPLE

A rowing boat: the oar pushes water back, the boat goes forward.



HOW IT FLIES

Thrust against weight decides up or down

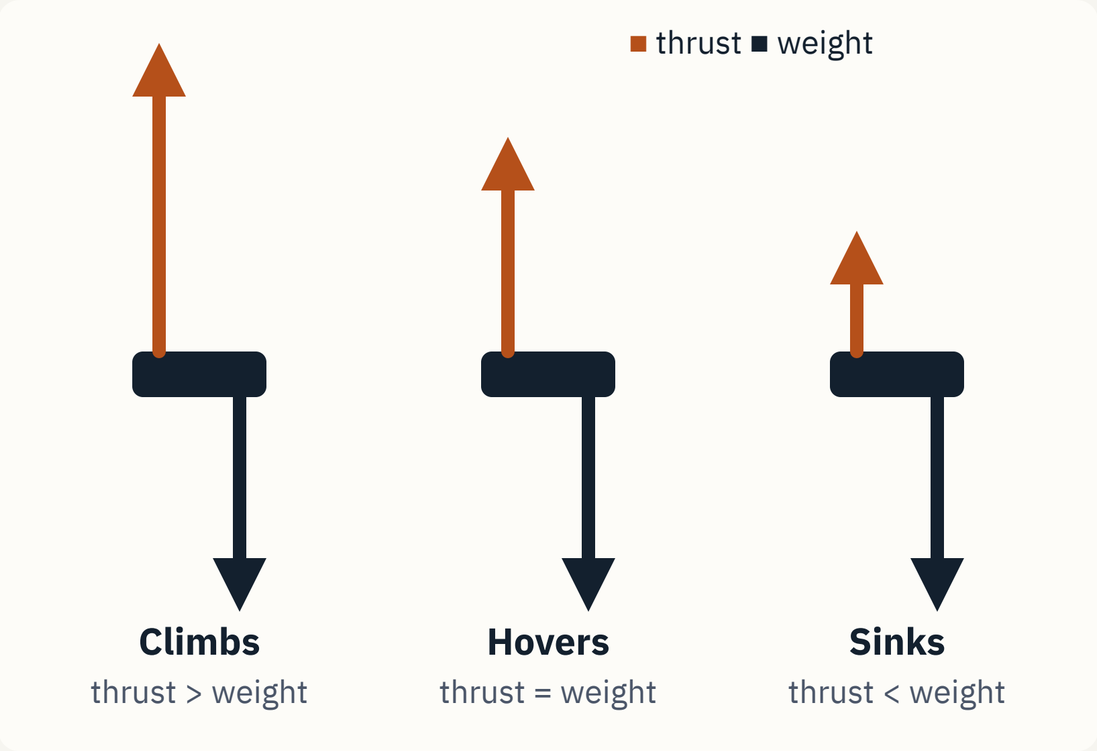
Up or down is a tug-of-war between thrust and weight.

Throttle up → climbs. Throttle in the middle → **hovers**. Throttle down → sinks.

As the battery drains, the same setting gives less thrust — so a fixed setting slowly sinks.

EXAMPLE

A 40 g drone needs about 40 g of thrust to hover: roughly 10 g from each motor (illustration).



Three ways to rotate: roll, pitch, yaw

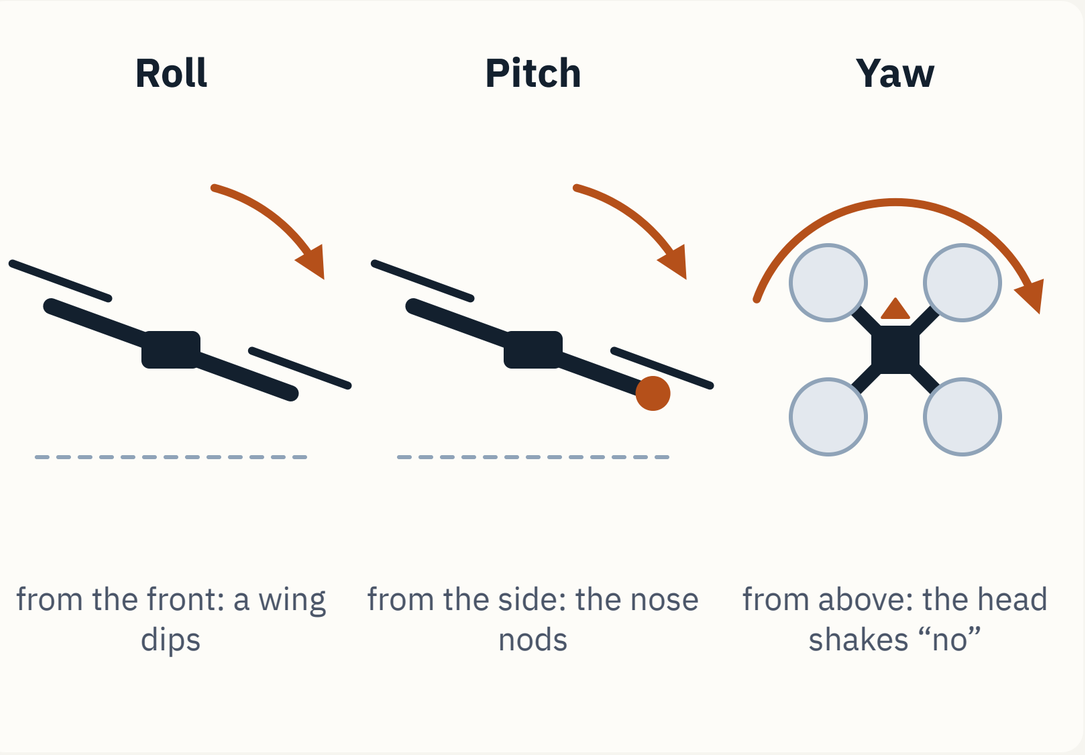
Three ways to rotate, plus throttle.

Roll moves the drone sideways.

Pitch moves it forward or back.

Yaw changes where the nose points.

Plus **throttle** for up and down: four controls, and one PID controller per rotation.



HOW IT FLIES

Two props spin one way, two spin the other

A spinning prop twists the drone the other way. Mixing directions cancels the twist.

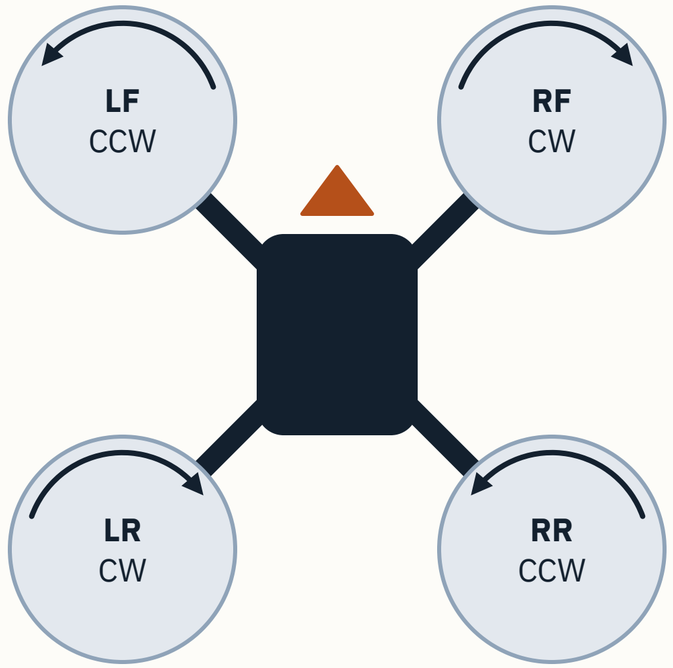
Clockwise: front-right (RF) and rear-left (LR).

Counter-clockwise: rear-right (RR) and front-left (LF).

Each pair sits on a diagonal, so the twists cancel and the drone stays balanced.

EXAMPLE

Two ceiling fans hanging loose, spinning opposite ways, would cancel each other's twist.



HOW IT FLIES

Roll: make one side push harder

To roll right, the left motors speed up and the right motors slow down.

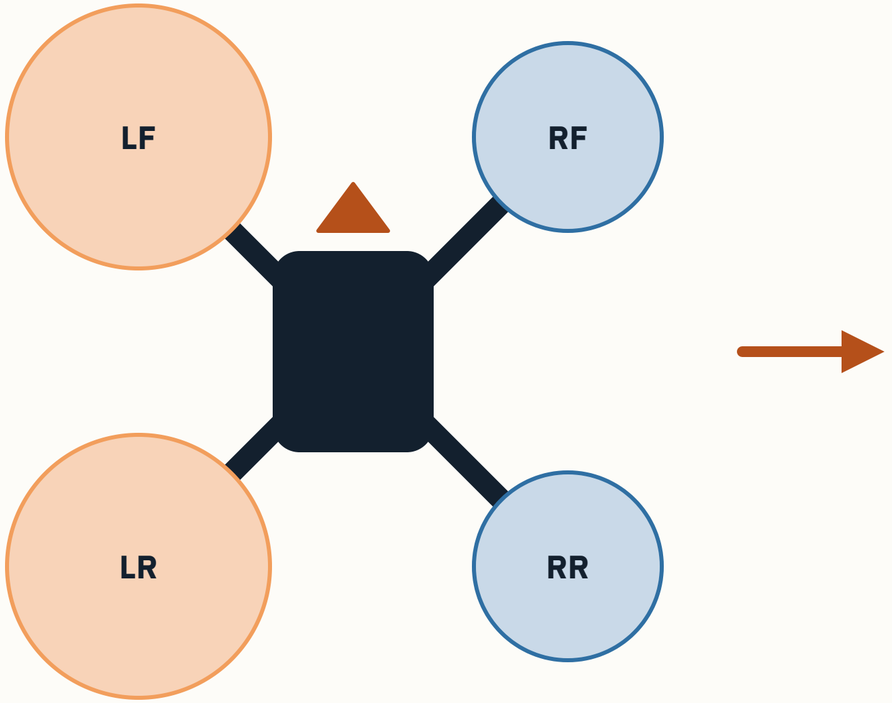
Orange, bigger = faster: LF and LR push harder, the left side rises.

Blue, smaller = slower: RF and RR push less, the right side drops.

Result: the drone tilts right and slides right →

EXAMPLE

Hover at 50% on all four. Roll right: LF, LR → 55%; RF, RR → 45%. Average still 50%.



HOW IT FLIES

Pitch: front against back

To fly forward, push harder at the back.

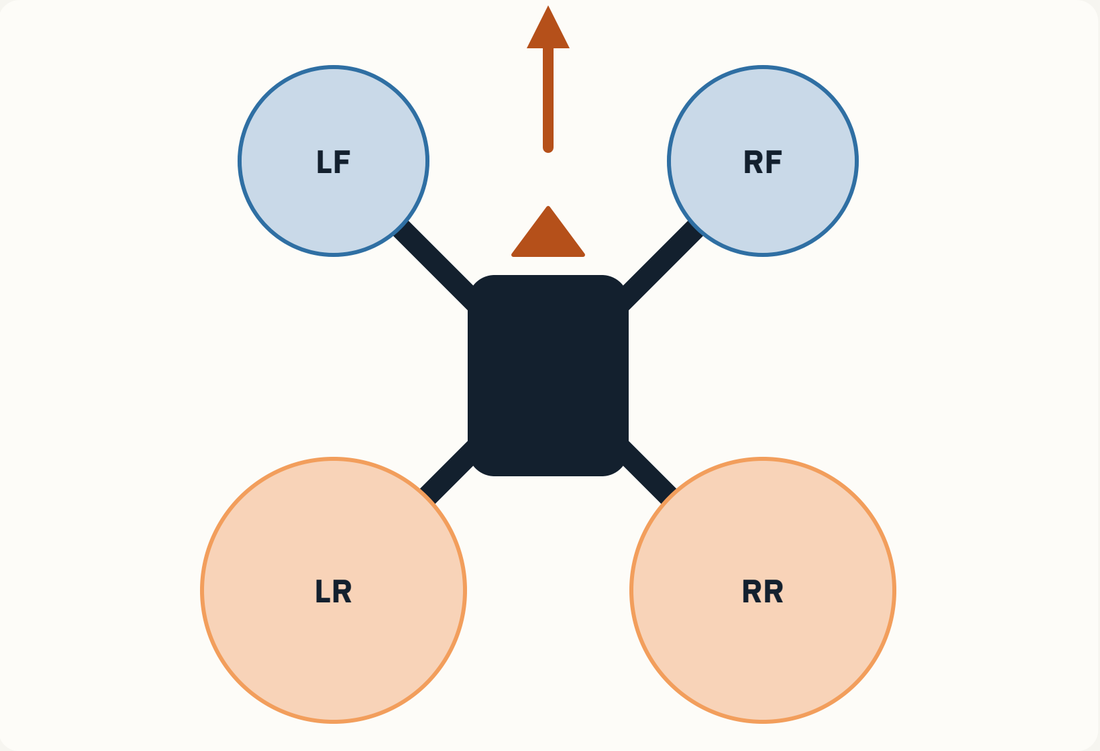
Rear (RR, LR) faster → the tail rises.

Front (RF, LF) slower → the nose drops.

Thrust now points a little forward, so the drone moves forward ↑

EXAMPLE

Leaning forward on a bicycle to start rolling: tilt first, then move.



HOW IT FLIES

Yaw: use the twist on purpose

To turn, use the twist on purpose.

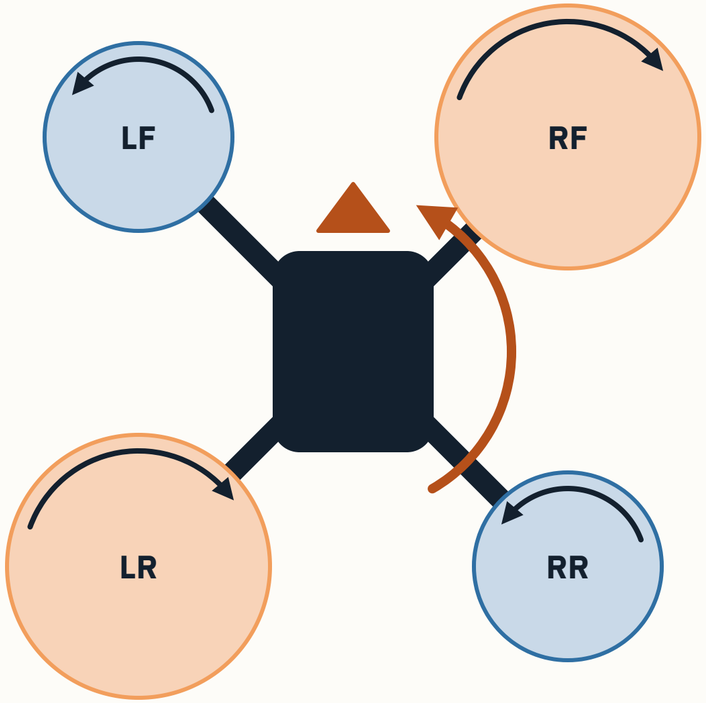
Speed up the **CW pair (RF, LR)** and slow the **CCW pair (RR, LF)**.

The twists no longer cancel: the body turns the other way (orange arrow).

Total thrust is unchanged, so height is kept.

EXAMPLE

On an office chair, swing your arms one way: your body turns the other way.



HOW IT FLIES

Summary: which motors change for each move

Move	Faster	Slower	Stays the same
Climb	all four	—	attitude
Roll right	LF, LR	RF, RR	total thrust
Pitch forward (nose down)	RR, LR	RF, LF	total thrust
Yaw (CW pair faster)	RF, LR	RR, LF	total thrust

Every move is “add here, take away there”. Only climbing changes all four the same way.

Why a quadcopter cannot fly by itself

A quadcopter is unstable: a small tilt grows in under a second.

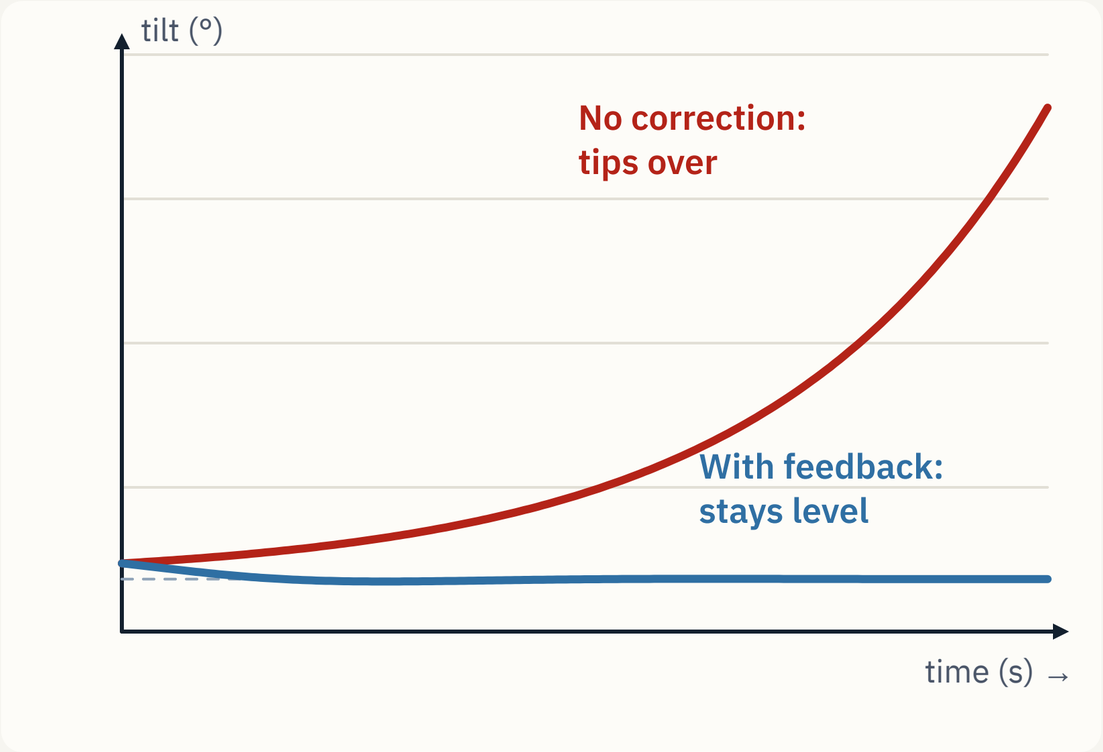
Motors are never equal: same setting, slightly different thrust.

The air moves: a breeze or the drone's own wash pushes one side.

Tilt feeds itself: a tilted drone slides, slides faster, tips over.

EXAMPLE

Balancing a broom on your hand: stop correcting and it falls. (Curves are an illustration.)



HOW IT FLIES

The fix is feedback

Example: keeping a car in its lane. You see the drift, compare it to the lane centre, and turn the wheel a little — over and over.

1. **Measure** what the drone is doing now (the motion sensor).
2. **Compare** it to what we want (the setpoint from the stick or the app).
3. **Correct**: change the motors by an amount based on the difference (the error).
4. **Repeat** — fast enough that errors stay small.

HOW IT FLIES

One control loop, 250 times every second

The same four steps, 250 times every second.

Sense: read gyro and accelerometer over SPI.

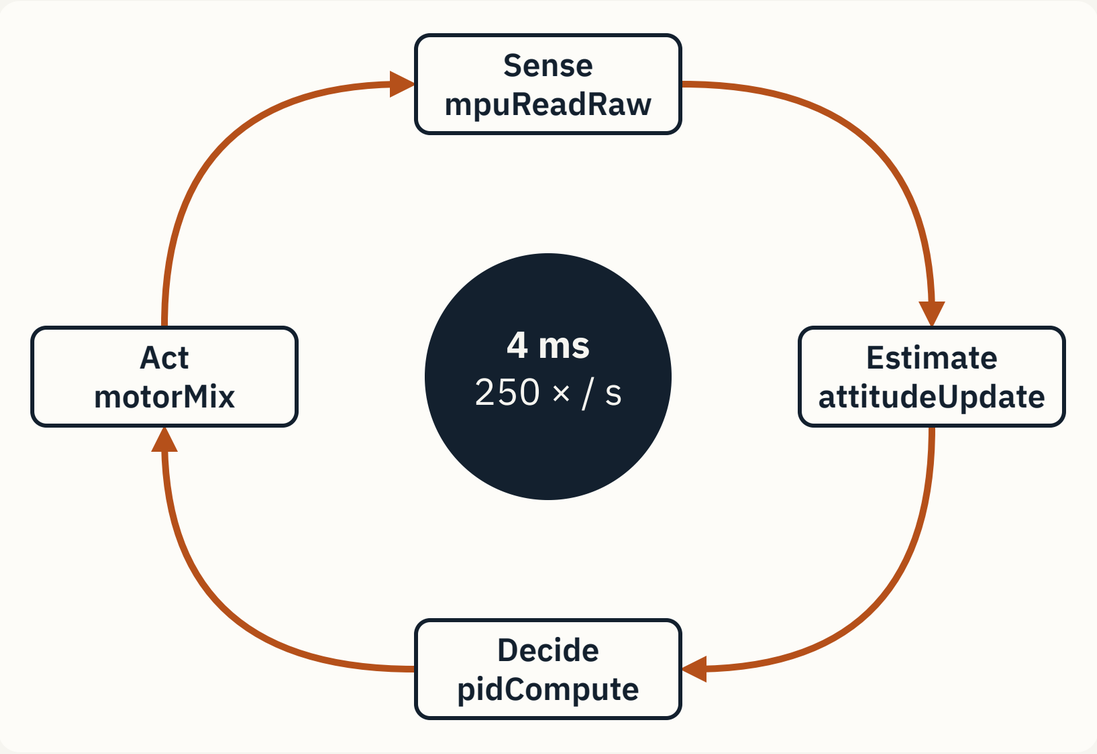
Estimate: work out the tilt angles.

Decide: the PID compares want with have.

Act: the mixer sets four motor duties — then wait until 4 ms have passed.

EXAMPLE

Gyro says 6 °/s roll → angle 2.1° → PID wants 0 → left motors +8, right −8 → wait.



HOW IT FLIES

Why 250 times a second?

4 ms

per loop (LOOP_TIME_US 4000). Short enough to catch a tilt before it grows.

250 Hz

the sensor sample rate is set to match: 1 kHz ÷ (1 + 3).

≈ 0.5°

an example drift between two loops — small, so each correction is small.

Rule of thumb: the loop must run much faster than the drone can tip over.

Check your understanding

Why are the props mixed CW and CCW?

Hint: think about the twist each prop puts on the body.

Why can a quadcopter only move sideways by tilting?

Hint: which way does thrust point?

Why does yaw need a bigger gain than roll?

Hint: torque from small props is weak.

Why must the loop be fast?

Hint: how quickly does a small tilt grow?

03

The hardware parts

Every part on the board, what it does, and how the firmware talks to it.

HARDWARE

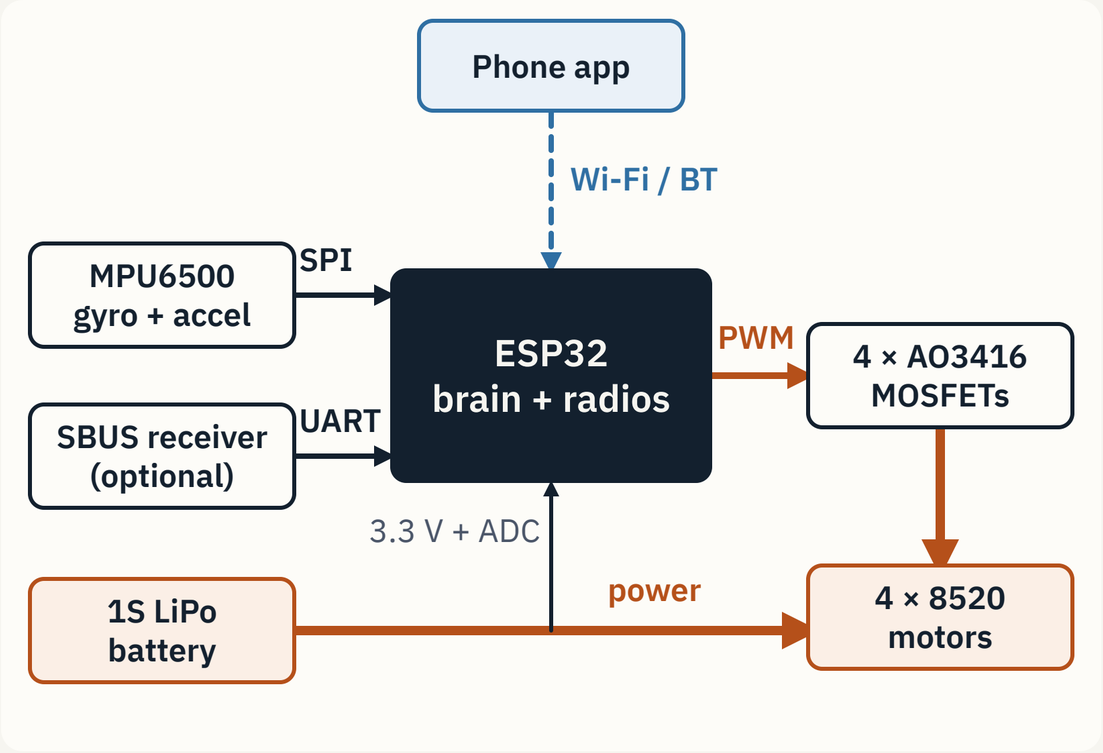
The board in one picture

One chip in the middle; everything else plugs into it.

Inputs (left): the motion sensor on SPI, an optional RC receiver on UART, the battery voltage on the ADC.

Outputs (right): PWM to four MOSFET switches, which drive four motors.

Radio (top): the phone talks to the ESP32 over Wi-Fi or Bluetooth.



HARDWARE

ESP32: the brain

Two cores, up to 240 MHz

Plenty of speed for a 250 Hz loop with floating-point maths.

Wi-Fi and Bluetooth built in

The phone connects directly — no extra radio chip.

Hardware helpers

SPI, UARTs, LEDC PWM and an ADC do the timing work in hardware.

EXAMPLE

Like a phone that is also its own router: it computes and talks without extra boxes.

HARDWARE

MPU6500: the motion sensor

One small chip with two sensors inside, each measuring three axes.

Gyroscope

How fast the drone is rotating, in degrees per second. Range set to ±500 °/s.

Accelerometer

The push it feels, including gravity. Range set to ±8 g.

SPI connection

The ESP32 reads 14 bytes in one burst every loop.

HARDWARE

Gyro and accelerometer: each fixes the other

Each sensor is wrong in a different way.

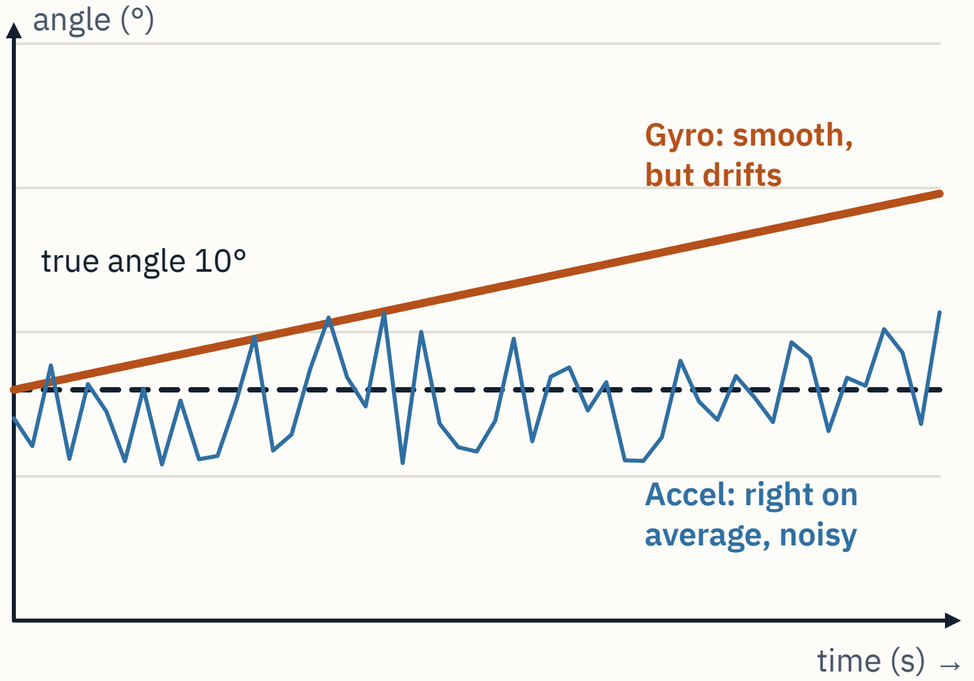
Gyroscope: fast and smooth, but a small bias adds up — the angle drifts.

Accelerometer: knows where “down” is and never drifts, but vibration makes it noisy.

Section 5 combines them so each covers the other's weakness.

EXAMPLE

Gyro = a stopwatch (precise, drifts). Accelerometer = the sun (always right, hard to read on a cloudy day).



HARDWARE

SPI: four wires to the sensor

Four wires, one direction each.

SCK: the clock — the ESP32 sets the pace.

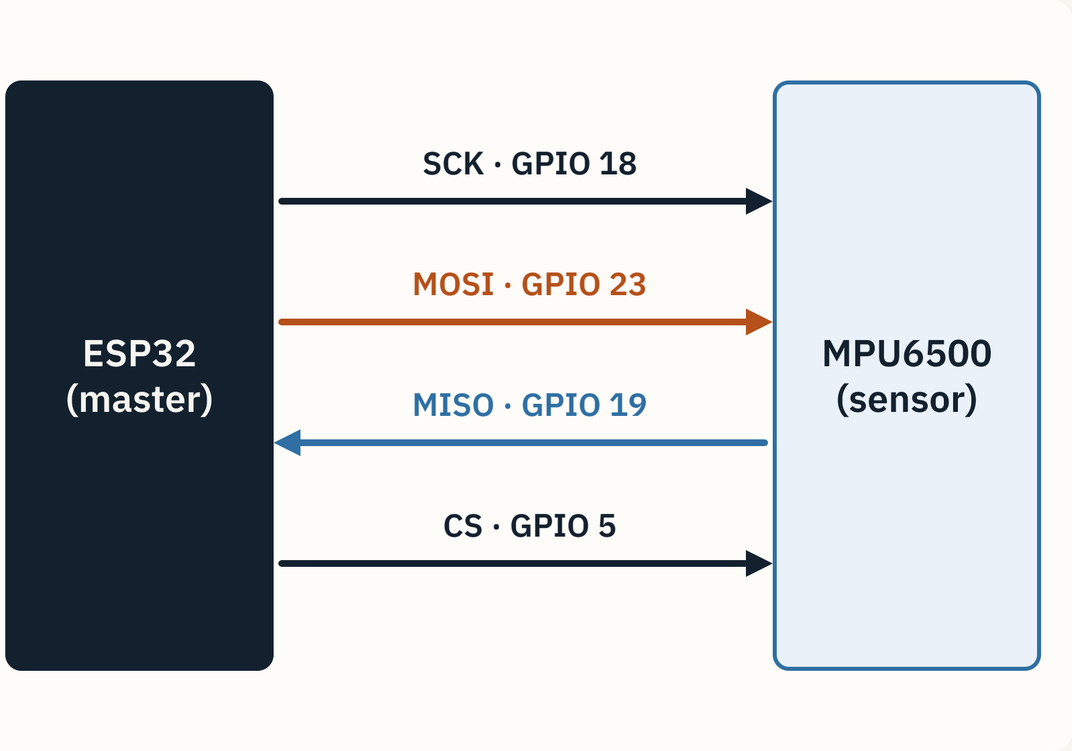
MOSI: data out to the sensor. **MISO**: data back.

CS: LOW means “I am talking to you”.

This firmware runs SPI at 1 MHz, mode 0.

EXAMPLE

Reading 14 bytes at 1 MHz takes about 0.12 ms — 3% of the 4 ms loop.



HARDWARE

SBUS: an optional RC receiver

A hobby radio receiver can plug in on GPIO 4 and fly the drone with a real transmitter.

One signal wire

16 channels packed into one 25-byte frame.

Unusual serial

100000 baud, 8 data bits, even parity, 2 stop bits, inverted.

The safety net

SBUS always flies in manual rate mode. Section 8 decodes it.

EXAMPLE

Serial2.begin(100000, SERIAL_8E2, PIN_SBUS_RX, -1, true); — the final true turns on the inversion.

HARDWARE

AO3416 MOSFET: the motor switch

A GPIO pin cannot power a motor. A MOSFET is an electronic switch that can.

Small signal in

The ESP32 pin drives the gate with 3.3 V and almost no current.

Big current through

The MOSFET lets battery current flow through the motor.

Fast

It can switch on and off 20,000 times a second.

EXAMPLE

Like a light switch you flip with one finger that controls a big lamp.

HARDWARE

8520 coreless brushed motors

The name is the size: 8.5 mm wide, 20 mm long.

Brushed

Spin faster with more average voltage. No ESC needed.

Coreless

Light rotor, so speed changes quickly — good for control.

Driven by duty

The firmware sets how much of each cycle the MOSFET is on.

HARDWARE

PWM: fast on-off makes an average

On-off, very fast. The motor feels the average.

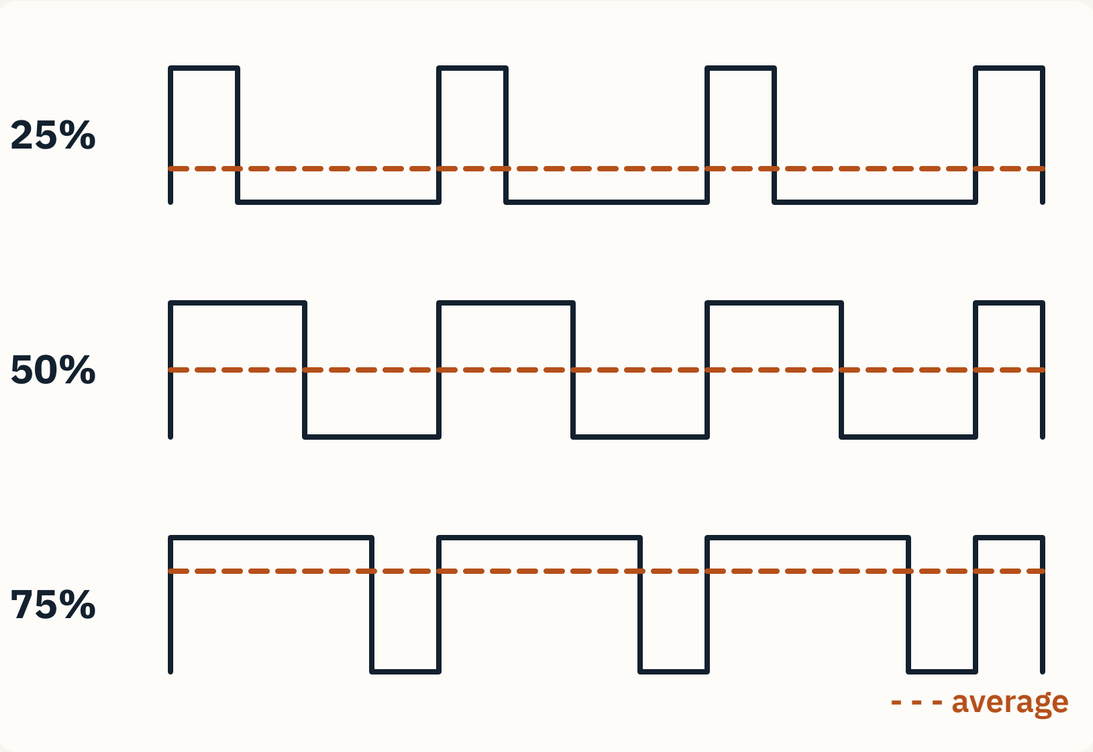
Duty cycle: the fraction of each cycle the switch is ON.

20 kHz: one cycle every 50 µs — too fast for the motor to notice.

10 bits: duty is a number from 0 to 1023.

EXAMPLE

Battery 4.0 V, duty 256 of 1023 (25%) → the motor feels about 1.0 V.



HARDWARE

The battery: one LiPo cell (1S)

One LiPo cell (1S): voltage is the fuel gauge.

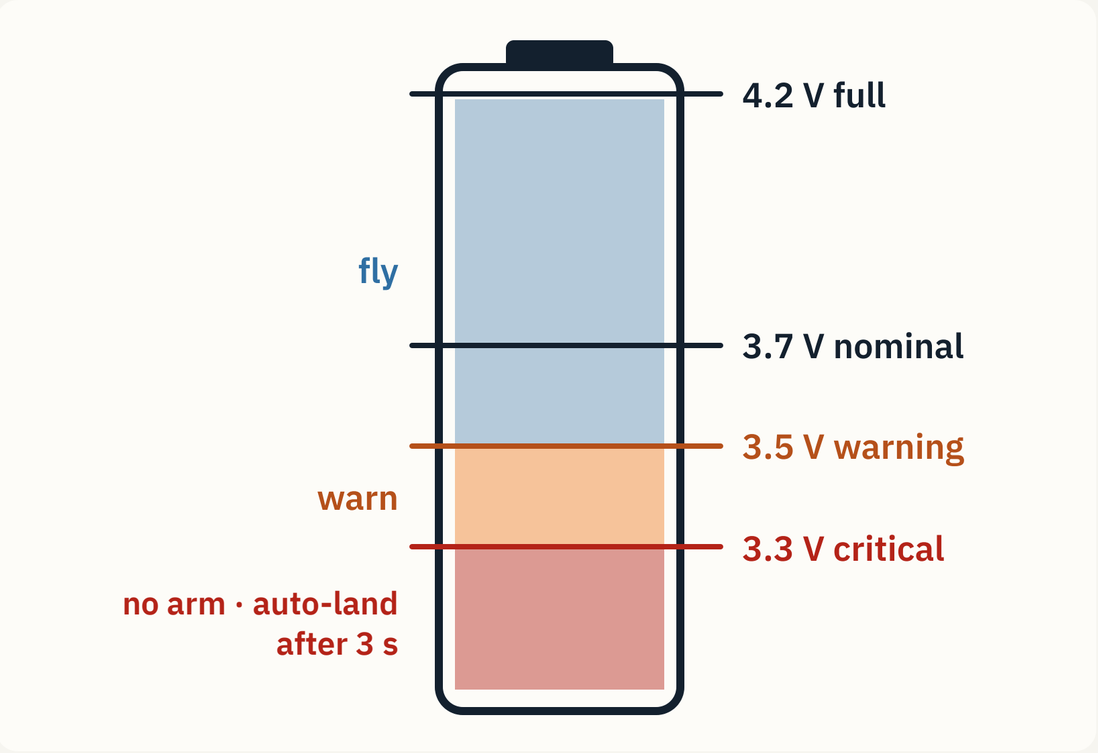
A full cell is **4.2 V**.

Below **3.5 V** the firmware warns (VBAT_CELL_WARN).

Below **3.3 V** it refuses to arm, and lands itself if it stays there for 3 s (VBAT_CELL_CRIT).

EXAMPLE

Thrust also drops with voltage — that is why hover height sinks during a flight.



HARDWARE

Measuring the battery: a voltage divider

Two resistors scale the battery down to a safe level.

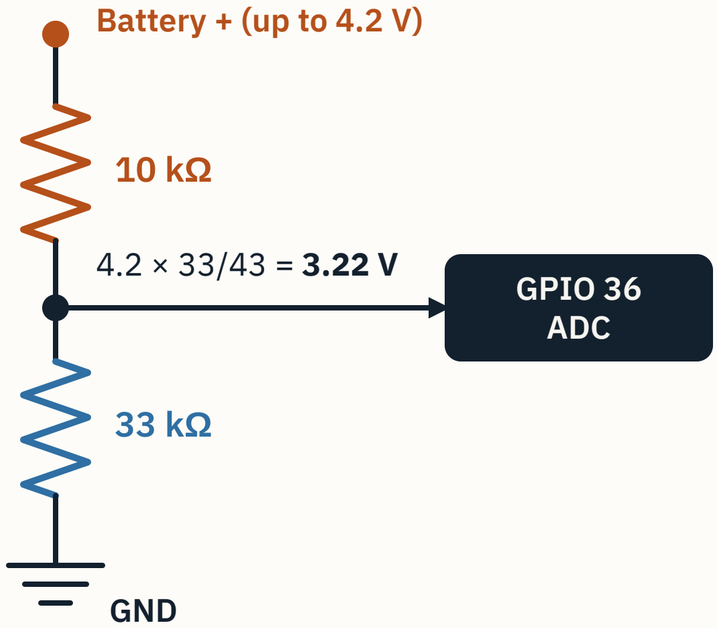
The ADC pin reads up to about 3.3 V, but a full cell is 4.2 V.

Pin voltage = battery × 33 ÷ 43. The firmware multiplies back by **43 ÷ 33** (VBAT_DIVIDER).

Change the resistors → change VBAT_DIVIDER too.

EXAMPLE

The pin reads 3.00 V → battery = 3.00 × 43 ÷ 33 = 3.91 V.



HARDWARE

Reading and smoothing the battery

```
uint32_t mv = analogReadMilliVolts(PIN_VBAT);
float sample = (mv / 1000.0f) * VBAT_DIVIDER;
if (vbat ⩽ 0.0f) vbat = sample;          // first
read
else vbat += VBAT_FILTER_ALPHA * (sample - vbat);
```

analogReadMilliVolts uses the chip's factory calibration.

Alpha 0.02: each loop moves 2% toward the new reading — about a 0.2 s time constant.

Why smooth? One reading is noisy and dips with every throttle punch.

EXAMPLE

Old value 3.90 V, one noisy sample 3.60 V → new value 3.90 + 0.02 × (−0.30) = 3.894 V. One bad reading barely moves it.

HARDWARE

The pin map

Pin	Use	Pin	Use
GPIO 15	Motor RF (front-right)	GPIO 18	SPI SCK
GPIO 13	Motor RR (rear-right)	GPIO 19	SPI MISO
GPIO 12	Motor LR (rear-left)	GPIO 23	SPI MOSI
GPIO 14	Motor LF (front-left)	GPIO 5	MPU chip select
GPIO 4	SBUS in (UART2 RX)	GPIO 36	Battery ADC

HARDWARE

Motor layout, seen from above

Seen from above, nose up.

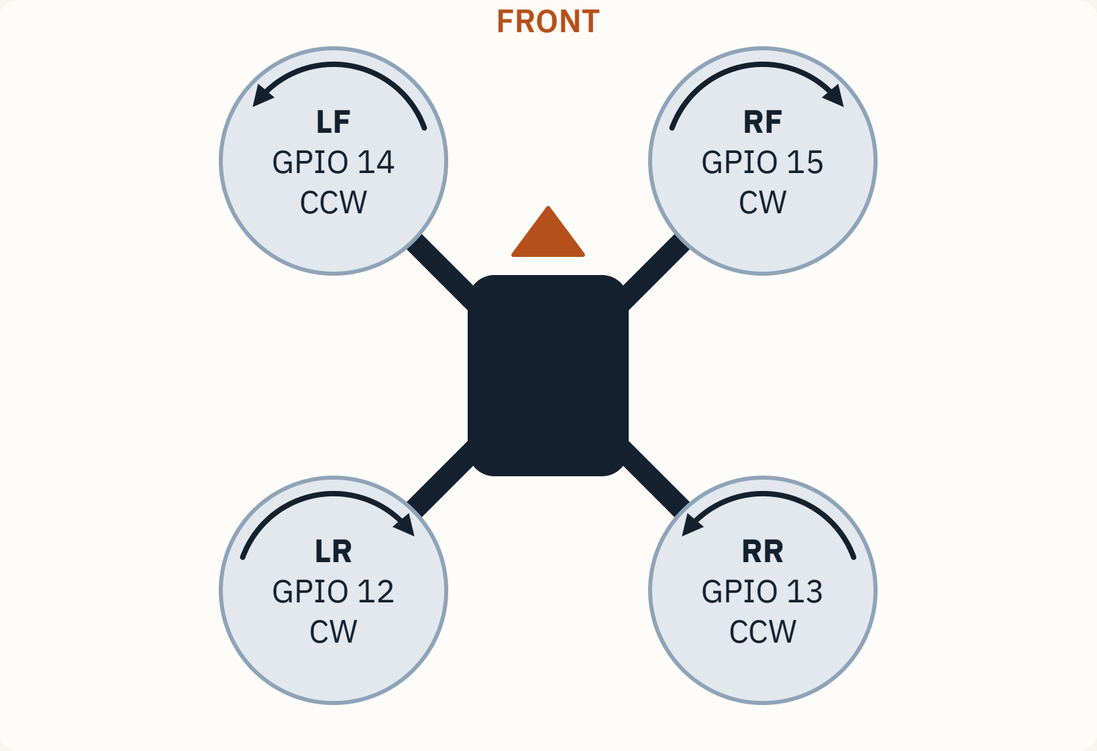
An **X frame**: the nose points between the two front motors.

Diagonals match: RF and LR clockwise; LF and RR counter-clockwise.

Names are from the drone's point of view, looking forward.

EXAMPLE

Props off, arm, gently roll it by hand: the low side's motors should speed up.



HARDWARE

If the props are reversed: YAW_DIRECTION

Some builds put CW props on RR and LF instead. Then every yaw correction pushes the wrong way.

Symptom

The drone spins faster and faster as soon as it lifts off.

Fix

Change #define YAW_DIRECTION from 1 to –1.

Why it works

The mixer multiplies the yaw output by YAW_DIRECTION, flipping all four yaw signs.

HARDWARE

The power path

One battery feeds three paths. Line thickness = current.

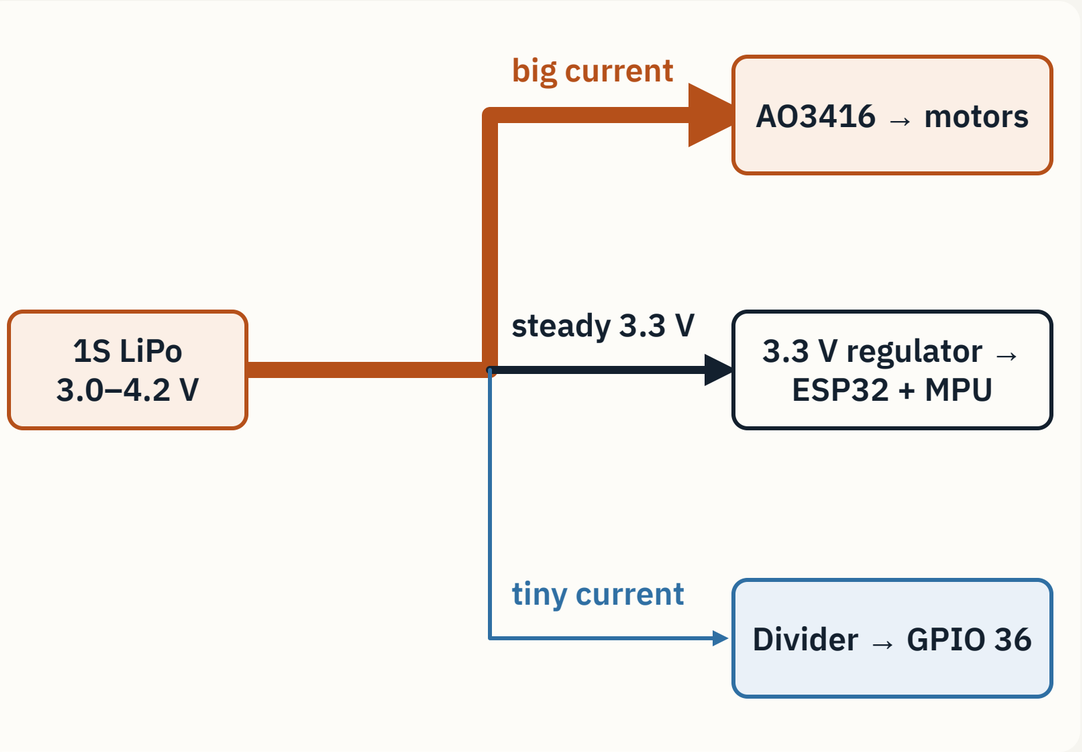
Motors take most of the current, straight from the battery.

Logic needs a steady 3.3 V, so it goes through a regulator.

The divider lets the firmware watch the voltage.

EXAMPLE

Full-throttle punch: the battery sags 3.9 → 3.6 V for a moment; the ESP32 still sees a steady 3.3 V.



HARDWARE

What is NOT on the board — and why it matters

No barometer

Cannot measure height. Hold keeps a thrust level, not an altitude.

No compass

Heading comes from the gyro only, so it drifts slowly.

No GPS

Cannot know its position. Follow uses the phone's GPS and dead reckoning.

No rangefinder

Cannot see the ground. Landing is a timed thrust ramp.

HARDWARE

Component summary

Part	Job	Connection	Main code
ESP32	runs everything	—	setup(), loop()
MPU6500	rotation + gravity	SPI	mpuInit(), mpuReadRaw()
SBUS receiver	RC sticks (optional)	UART2 RX	sbusPoll(), sbusDecode()
AO3416 × 4	motor switches	LEDC PWM	motorInit(), motorWrite()
8520 × 4	thrust	via MOSFETs	motorMix()
Divider	battery voltage	ADC GPIO 36	batteryUpdate()

HARDWARE · ASK WHY

Check your understanding

Why does a GPIO pin need a MOSFET to drive a motor?

Hint: compare the pin's current with the motor's.

Why is PWM at 20 kHz and not 1 kHz?

Hint: listen to the motors.

Why must the battery be divided before the ADC?

Hint: what is the ADC's top voltage?

Why can't the drone hold its height?

Hint: which sensor would measure height?

04

Firmware architecture

One Arduino sketch, one loop, and the order things happen in.

ARCHITECTURE

One sketch, two entry points

Every Arduino program has the same shape. Esp32_drone.ino is no different.

setup()

Runs once at power-on. Starts the hardware, loads saved settings, calibrates the gyro.

loop()

Runs forever, once every 4 ms. Reads, decides, drives the motors.

No RTOS tasks

The flight code does not create its own threads. One loop, one order.

EXAMPLE

Like a cook following one recipe card from top to bottom, again and again, instead of several cooks working at once.

ARCHITECTURE

What setup() does, in order

1. **motorInit()** and **motorStop()** — motors off before anything else.
2. **mpuInit()** — if WHO_AM_I is wrong, print a wiring hint and stop forever.
3. ADC attenuation for the battery pin, then **sbusInit()**.
4. Open NVS and **cfgLoad()** — saved PID gains and flight settings.
5. **linkInit()** (Wi-Fi / Bluetooth), then **mpuCalibrateGyro()** and a first battery read.

ARCHITECTURE

Map of the sketch

Section	Main functions	Course section
Config #defines	pins, rates, gains, limits	3, 4
MPU6500	mpuInit, mpuReadRaw, mpuCalibrateGyro	5
SBUS	sbusInit, sbusDecode, sbusPoll	8
Transport + CRTP	linkInit, linkPoll, crtpDispatch, handlers	8
Motors + battery	motorInit, usToDuty, motorWrite, batteryUpdate	3, 7
PID + attitude	pidCompute, attitudeUpdate	5, 6
Navigation	navUpdate, navSetpoints	10
Modes + arming	flightSetpoints, armUpdate, motorMix	6, 7, 9

ARCHITECTURE

Global state: the variables everyone shares

Variable	Meaning	Written by
armed	motors allowed to spin	armUpdate, flight commands, failsafe
flight_mode, auto_state	MANUAL…RTH, takeoff/hold/land	port 7 commands
gyro_roll / pitch / yaw	rotation rates, °/s	attitudeUpdate
angle_roll_deg / pitch	filtered tilt angles, °	attitudeUpdate
sp_* setpoints	what we want this loop	flightSetpoints
motor_rf … motor_lf	1000–2000 command per motor	motorMix

ARCHITECTURE

loop() in twelve lines

```
sbusPoll();  linkPoll();              // 1. receive
// 2. decide who is in control, failsafe
if (!rx_ok && armed) { armed = false; motorStop(); }
armUpdate(now, use_crtp);             // 3. arm / disarm
mpuReadRaw();  attitudeUpdate();      // 4. sense
navUpdate(...);                       // 5. dead reckoning
flightSetpoints(now, use_crtp, sbus_ok);  // 6. want
roll_setpoint = sp_roll_rate - roll_level_adjust;
pidCompute();                         // 7. decide
motorMix(sp_throttle_us);             // 8. mix
motorWrite(motor_rf, motor_rr, motor_lr, motor_lf);
batteryUpdate(); logUpdate(); telemetry(now);
while (micros() - loop_timer < LOOP_TIME_US) { }
```

Shortened — the real loop has comments and checks between these lines.

Order matters: safety checks come before sensing and control, so a lost link stops the motors in the same loop.

ARCHITECTURE

How data flows through one loop

Inputs
SBUS sticks or phone CRTP

→

Setpoints
flightSetpoints(): target angles + rates

→

Self-level
angle error → rate setpoint

→

PID
rate error → correction

→

Mixer
corrections → 4 motors

→

PWM
duty to MOSFETs

Sensor data joins twice: angles at self-level, gyro rates at the PID.

EXAMPLE

Stick right → sp_roll_rate +50 °/s → gyro says 0 → error −50 → PID output → right/left motors change → the drone starts to roll.

ARCHITECTURE

Hard 250 Hz pacing

```
// ---- hard 250 Hz pacing ----
while (micros() - loop_timer < LOOP_TIME_US) { /*
spin */ }
loop_timer = micros();
```

Busy-wait: the CPU spins until exactly 4 ms have passed since the last loop.

Why exact timing? The gyro is integrated with a fixed dt of 0.004 s. A loop of the wrong length gives the wrong angle.

Wrap-safe: unsigned subtraction still works when micros() overflows after ~71 minutes.

EXAMPLE

The work takes 1.3 ms this loop → the spin lasts 2.7 ms. Next loop the work takes 2.1 ms → the spin lasts 1.9 ms. The period stays 4 ms.

ARCHITECTURE

Inputs: who can fly the drone

Phone app (CRTP)

Over Wi-Fi UDP, Bluetooth Classic SPP or BLE. Wins whenever its link is live.

RC receiver (SBUS)

Optional. Used when the phone is silent. Always manual rate mode.

Serial console

USB cable, for bench setup: link, sta, save, load, defaults, status.

EXAMPLE

Phone connected and sending → phone flies. Phone app closed for 0.5 s → SBUS takes over if a receiver is fitted; if not, failsafe stops the motors.

ARCHITECTURE

Outputs: what the firmware produces

Motor PWM

Four duty values, every 4 ms. The only output that moves the drone.

CRTP replies + logs

Acks for commands, parameter values, and up to 3 log streams to the phone.

Serial telemetry

One human-readable line per second: VBAT, sticks, angles, motors.

ARCHITECTURE

Configuration lives in #defines

#define	Value	Changes
LOOP_TIME_US	4000	loop period (250 Hz)
PID_ROLL_KP / KI / KD	1.3 / 0.04 / 18	default roll gains
MAX_TILT_DEG	25	largest commanded tilt
LEVEL_SENSITIVITY	15	how hard self-level pulls back
FAILSAFE_MS	500	silence before motors stop
VBAT_CELL_CRIT	3.30	no-arm and auto-land voltage

ARCHITECTURE

Saving settings: NVS flash

```
void cfgSave() {
  float g[9] = { pid_p_gain_roll, ... };   // 9
gains
  uint16_t f[3] = { hover_thrust, takeoff_ms,
land_ms };
  nvs.putBytes("pid",  g, sizeof(g));
  nvs.putBytes("fcfg", f, sizeof(f));
}
```

NVS = non-volatile storage: a small key-value store in the ESP32 flash.

Raw floats: a read-back is byte-identical to what was written — no rounding.

Refused while armed: erasing flash stalls both cores for milliseconds.

EXAMPLE

Tune roll_kp to 1.5 on the phone, land, press Save → it survives the next power-off.

ARCHITECTURE

Link profiles: how the phone connects

#	Profile	Good for
0	Wi-Fi access point (drone is the hotspot)	default; simplest
1	Wi-Fi station (joins your router)	phone stays on home Wi-Fi
2	Bluetooth Classic SPP	Android phones
3	Bluetooth Low Energy	iPhone (no SPP on iOS)
4	Wi-Fi AP + BLE	two ways in at once
5	Wi-Fi station + BLE	router plus BLE backup

ARCHITECTURE

Four design principles you will see everywhere

Safety first in the loop

Failsafe and arming run before control, every loop.

Honest about limits

No barometer → no fake altitude. Comments say so.

Commands are idempotent

The app resends every 500 ms, so applying a command twice must be safe.

Tests pin fragile lines

run.sh fails if a tested formula changes in the sketch.

ARCHITECTURE · ASK WHY

Check your understanding

Why are the motors stopped first in setup()?

Hint: what could a floating gate do?

Why use a busy-wait instead of delay()?

Hint: what happens to the period when work time changes?

Why must commands be safe to apply twice?

Hint: what does the app do when an ack is lost?

Why is SAVE refused while armed?

Hint: how long does a flash erase take?

05

Reading the motion sensor

From raw SPI bytes to a tilt angle you can trust.

MOTION SENSOR

Talking to a register over SPI

```
uint8_t mpuRead(uint8_t reg) {
  mpuSelect();                       // CS low
  SPI.transfer(reg | 0x80);          // bit7 = 1 → read
  uint8_t v = SPI.transfer(0x00);    // clock the answer out
  mpuDeselect();                     // CS high
  return v;
}
// write: reg & 0x7F (bit7 = 0), then the value
```

First byte = register address. Its top bit says read (1) or write (0).

SPI is a swap: to receive a byte you must send one, so we send 0x00.

CS low starts the conversation; **CS high** ends it.

EXAMPLE

Read WHO_AM_I (register 0x75): send 0xF5 (0x75 | 0x80), then 0x00, and the sensor answers 0x70 for an MPU6500.

MOTION SENSOR

Sensor set-up in mpuInit()

Register	Value	Meaning	Why
PWR_MGMT_1	0x80 then 0x01	reset, then use the gyro PLL clock	clean start; stable clock
SMPLRT_DIV	0x03	1 kHz ÷ (1+3) = 250 Hz	one fresh sample per loop
CONFIG	0x02	low-pass filter, 92 Hz	cuts motor vibration noise
GYRO_CFG	0x08	±500 °/s → 65.5 LSB per °/s	fast enough for flips, fine detail
ACCEL_CFG	0x10	±8 g → 4096 LSB per g	headroom for bumps

MOTION SENSOR

One burst read: 14 bytes, all axes

```
void mpuReadRaw() {
  uint8_t buf[14];
  mpuReadBurst(MPU_REG_ACCEL_XOUT, buf, 14);
  raw_ax = (int16_t)(buf[0]  << 8 | buf[1]);
  raw_ay = (int16_t)(buf[2]  << 8 | buf[3]);
  raw_az = (int16_t)(buf[4]  << 8 | buf[5]);
  /* buf[6..7] = temperature (skip) */
  raw_gx = (int16_t)(buf[8]  << 8 | buf[9]);
  raw_gy = (int16_t)(buf[10] << 8 | buf[11]);
  raw_gz = (int16_t)(buf[12] << 8 | buf[13]);
}
```

Registers 0x3B…0x48 sit next to each other, so one read gets everything.

Why one burst? All six values come from the same instant, and it is fast.

MOTION SENSOR

Two bytes make one signed number

Each value is 16 bits, sent high byte first. The cast to int16_t makes it signed.

Positive

0x01, 0x2C → 0x012C = 300

Negative

0xFF, 0x38 → 0xFF38 = −200 (two's complement)

Range

−32768 to +32767 for every axis

EXAMPLE

buf[8] = 0xFF, buf[9] = 0x38 → raw_gx = −200 → −200 ÷ 65.5 = −3.05 °/s.

MOTION SENSOR

From raw counts to real units

65.5

counts per °/s on the gyro (±500 °/s range)

4096

counts per g on the accelerometer (±8 g range)

0.004 s

dt: one loop, used to turn rate into angle

Example: raw gyro 655 → 10 °/s. Over one 4 ms loop that is 0.04°.

MOTION SENSOR

Gyro calibration: measure the bias at rest

```
int32_t sx = 0, sy = 0, sz = 0;
for (int i = 0; i < 2000; i++) {
  mpuReadRaw();
  sx += raw_gx; sy += raw_gy; sz += raw_gz;
  delay(3);
}
gyro_x_cal = sx / 2000;   // average = bias
gyro_y_cal = sy / 2000;
gyro_z_cal = sz / 2000;
```

A still gyro should read 0, but every chip reads a little off — the **bias**.

2,000 samples × 3 ms ≈ 6 seconds. The average is the bias, subtracted every loop.

Keep the drone flat and still or the bias will be wrong.

EXAMPLE

Bias 0.3 °/s not removed → after 60 s the heading is off by 0.3 × 60 = 18°.

MOTION SENSOR

Tilt from gravity: the accelerometer angle

The accelerometer finds the tilt from the share of gravity on each axis.

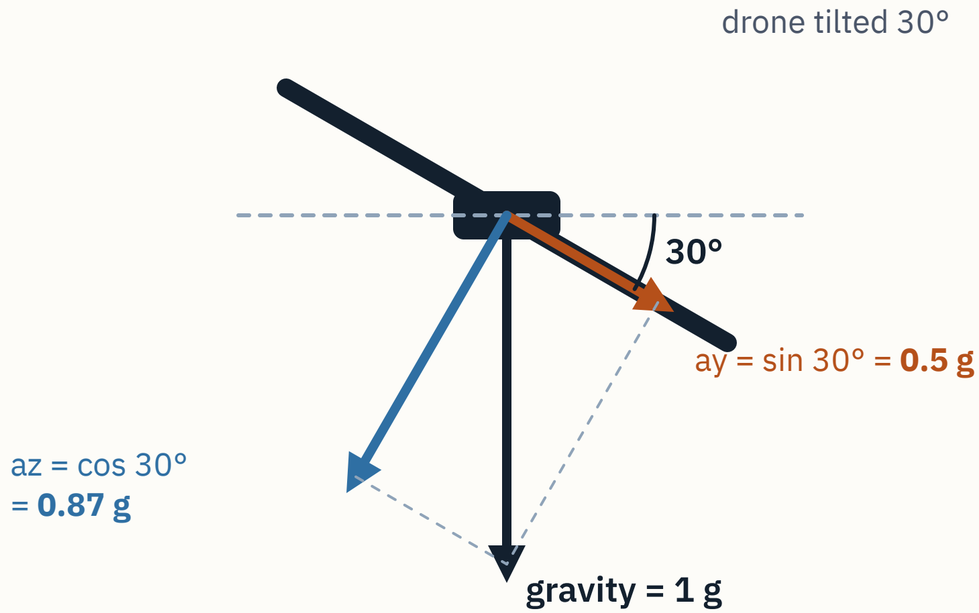
At rest the only force is gravity. Tilted, part of it falls on the flat axis.

asin(ay ÷ total) turns that share back into an angle.

Only true when the drone is not accelerating: a punch or a bump makes it jump.

EXAMPLE

Tilted 30°: ay = sin 30° × 1 g = 0.5 g → asin(0.5) = 30°.



MOTION SENSOR

The complementary filter idea

Combine them: mostly trust the gyro, and slowly pull toward gravity.

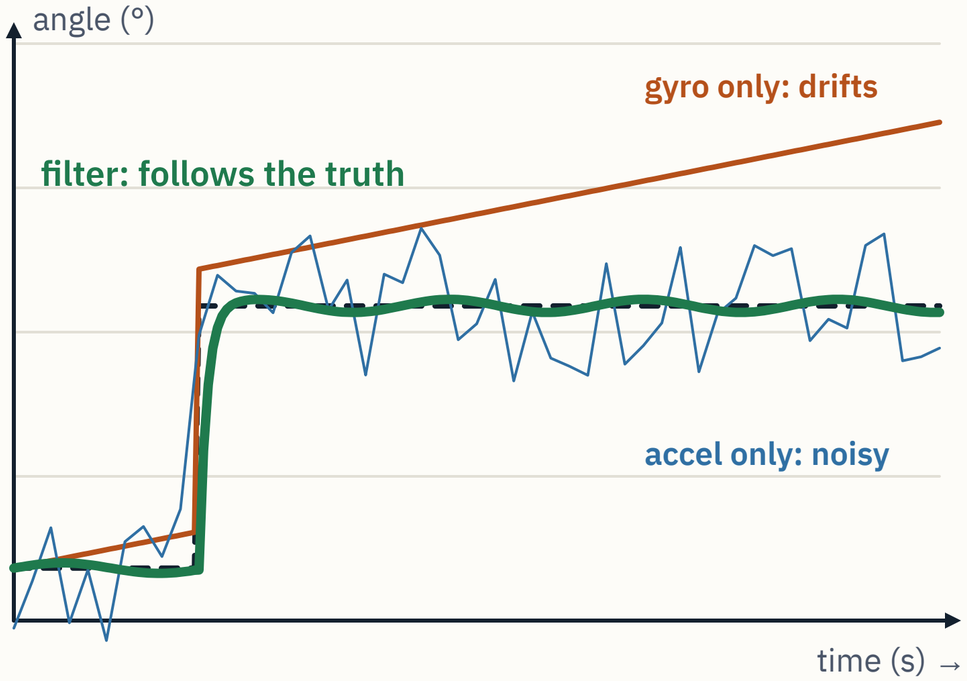
Each loop: move the angle by what the **gyro** says (fast, smooth).

Then nudge it 0.4% toward the **accelerometer** angle (slow, never drifts).

The **result** keeps the good half of each.

EXAMPLE

Curves are an illustration of the idea, not a recording.



MOTION SENSOR

The filter in code

```
angle_pitch_deg += gx * dt;      // 1. predict
with gyro
angle_roll_deg  += gy * dt;
angle_pitch_deg = angle_pitch_deg * 0.996f
                + angle_pitch_acc * 0.004f;   //
2. correct
angle_roll_deg  = angle_roll_deg  * 0.996f
                + angle_roll_acc  * 0.004f;
```

Step 1: move the angle by what the gyro says happened this loop.

Step 2: blend 99.6% of that with 0.4% of the accelerometer angle.

On the very first loop, the angle starts from the accelerometer (gyro_angles_set).

EXAMPLE

Angle 10.00°, gyro says +0.04° this loop, accel says 9.0° → (10.04 × 0.996) + (9.0 × 0.004) = 10.036°.

MOTION SENSOR

Why 0.996 and 0.004?

0.4%

of the accelerometer per loop: small, so vibration barely shows

≈ 1 s

time constant: 0.004 s ÷ 0.004 — how fast drift is pulled out

250

loops per second, so the 0.4% nudges add up quickly

Ask why: what would happen to these weights if the loop ran at 500 Hz?

MOTION SENSOR

Heading drifts — and the code says so

Roll and pitch have gravity to correct them. Heading (yaw) has nothing.

How it is made

heading_deg += gz × dt, wrapped to 0…360°.

Why it drifts

No magnetometer: any leftover gyro bias adds up forever.

What it affects

Heading hold in HOLD/AUTO and the frame for navigation.

EXAMPLE

Leftover bias 0.05 °/s → about 3° of heading drift per minute of flight.

MOTION SENSOR · ASK WHY

Check your understanding

Why does the SPI read byte have bit 7 set?

Hint: how does the sensor tell read from write?

Why calibrate the gyro at every power-on?

Hint: does bias change with temperature?

Why not use only the accelerometer for tilt?

Hint: what does it feel during a throttle punch?

Why can roll be corrected but heading cannot?

Hint: which direction does gravity point?

06

PID control

Turning “how wrong are we?” into “how hard do we push?”

PID CONTROL

PID in one analogy: a shower

P looks at now, I at the past, D at the future.

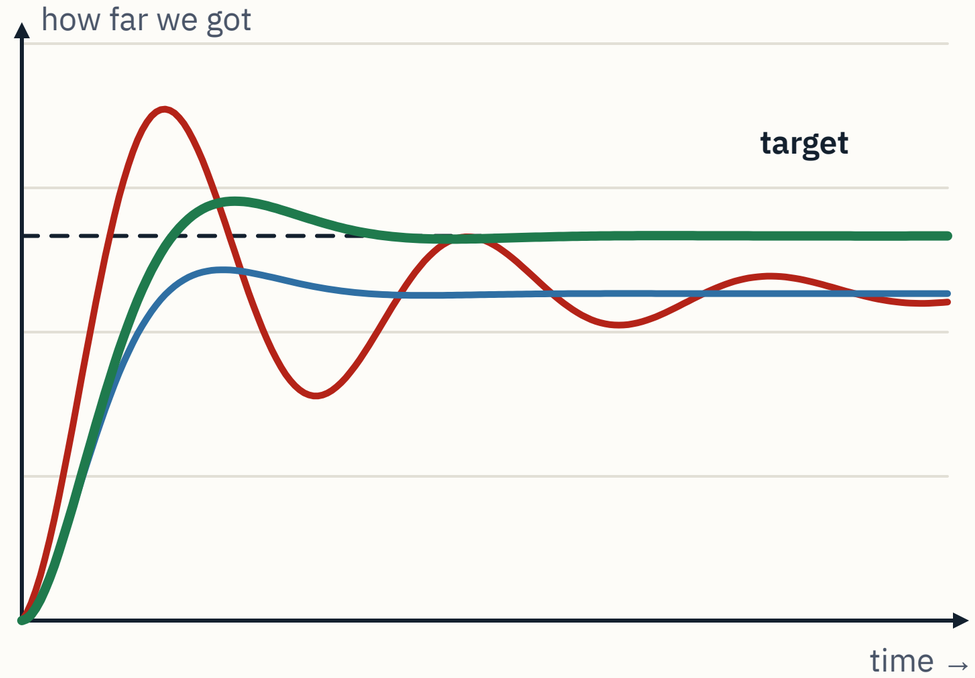
P only: pushes hard, overshoots, wobbles — and stops a little short.

P + D: D brakes the swing, so it is calm — still a little short.

P + I + D: I removes the last gap, so it settles on the target.

EXAMPLE

Shower: P turns toward hot, I keeps adding while it is still cold, D eases off when it heats up fast.



PID CONTROL

P: push in proportion to the error

P = Kp × error. Double the error, double the push.

Roll and pitch Kp

1.3

Yaw Kp

4.0 (yaw torque is weak)

Too much P

The drone shakes fast (oscillates). Too little: it feels soft and lazy.

EXAMPLE

The drone rolls at 10 °/s but we want 0 → error 10 → P = 1.3 × 10 = 13 units of motor correction.

I: fix the error that will not go away

I fixes the error that will not go away.

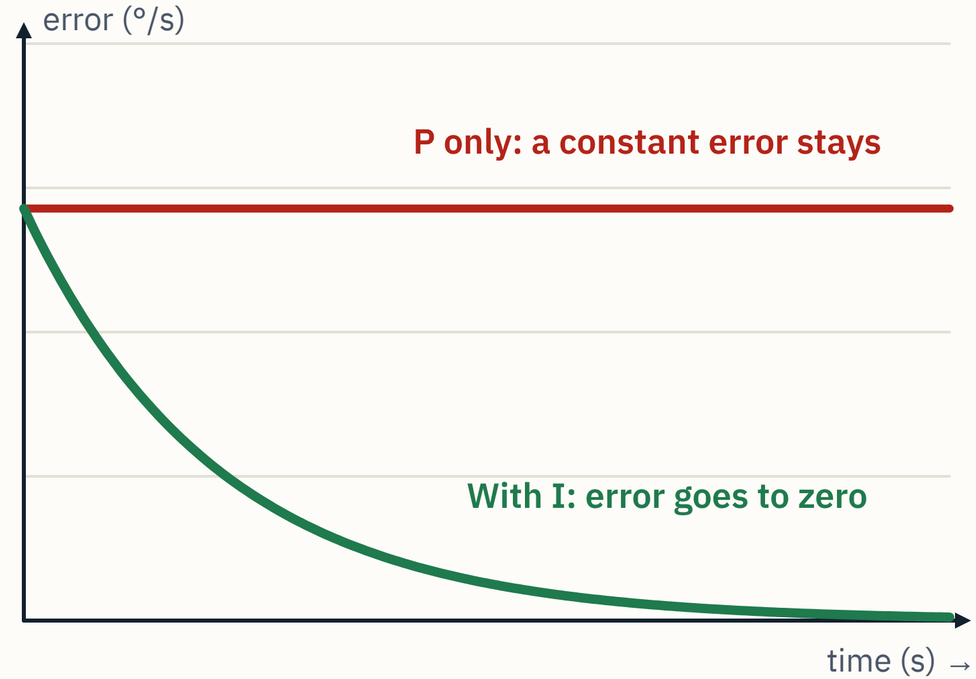
I adds up the error every loop: **i_mem += Ki × error**.

A small error that lasts becomes a big correction.

Roll and pitch Ki = 0.04 — tiny per loop, but 250 loops a second.

EXAMPLE

Battery 2 mm off-centre → error 1 °/s. After 1 s: I = 0.04 × 1 × 250 = 10 units — enough to hold level.



PID CONTROL

D: brake before you overshoot

D reacts to how fast the error is changing. It is a damper.

Code

D = Kd × (error – last error).

Roll and pitch Kd

18 — large, because the change per 4 ms loop is small.

Too much D

Hot motors and a high-pitched buzz: D amplifies sensor noise.

EXAMPLE

Error goes from 10 to 7 in one loop → change –3 → D = 18 × (–3) = –54, braking the correction before it overshoots.

PID CONTROL

pidCompute(): the roll axis

```
err = gyro_roll - roll_setpoint;
pid_i_mem_roll += pid_i_gain_roll * err;
pid_i_mem_roll = constrain(pid_i_mem_roll,
                     -I_TERM_LIMIT, I_TERM_LIMIT);
pid_p_roll = pid_p_gain_roll * err;
pid_i_roll = pid_i_mem_roll;
pid_d_roll = pid_d_gain_roll * (err - pid_last_roll_err);
pid_output_roll = pid_p_roll + pid_i_roll + pid_d_roll;
pid_last_roll_err = err;
pid_output_roll = constrain(pid_output_roll,
                     -PID_LIMIT, PID_LIMIT);
```

Pitch and yaw repeat the same ten lines with their own gains.

P, I and D are stored separately so the app can chart each term (log group **pid**).

PID CONTROL

The sign of the error matters

Here error = measured – wanted (gyro – setpoint). Many textbooks write wanted – measured.

Either works

...as long as the mixer signs match the choice.

The rule

A positive output must push the drone toward a negative rate.

If one sign is wrong

The correction becomes a push, and the drone flips at once.

EXAMPLE

Test with props off: tilt the drone by hand while armed. The motors on the low side must speed up. If the high side speeds up, a sign is wrong.

Wind-up: why I is limited and reset

Wind-up: I grows while it cannot act.

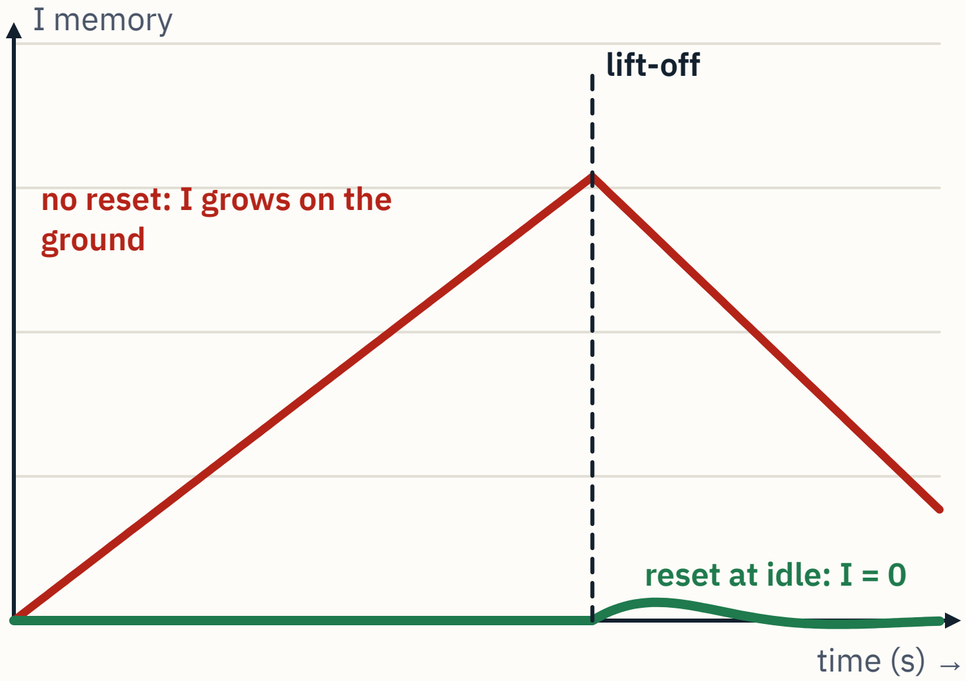
On uneven ground a small error never goes away, so I keeps growing.

At lift-off that stored I tips the drone hard.

Two fixes: clamp I to ±400, and reset it whenever the throttle is at idle.

EXAMPLE

Error 2 for 5 s on the ground → I = 0.04 × 2 × 1250 = 100 units stored before lift-off.



PID CONTROL

D on error, and its side effect

Both damp real motion the same way. They differ only when the setpoint jumps.

D on error (this firmware)

D = Kd × change in (gyro – setpoint)

Simple: one formula

A sudden stick move changes the setpoint → a one-loop spike (derivative kick)

D on measurement (alternative)

D = Kd × change in gyro only

No kick from stick moves

Slightly more code; the common choice in big flight stacks

Ask why the kick is small here: the stick setpoint changes smoothly and the output is clamped to ±400.

PID CONTROL

Limits that keep control possible

±400

PID output clamp (PID_LIMIT): one axis cannot take all the motor range

1800

THROTTLE_MAX: full stick leaves 200 of headroom for corrections

1100

armed minimum per motor: props never stop in the air

Example: throttle 1800, roll correction +250 → right motors want 2050 → clamped to 2000. Without the 1800 cap they would have no room at all.

Two loops stacked: angle outside, rate inside

Two loops stacked: angle outside, rate inside.

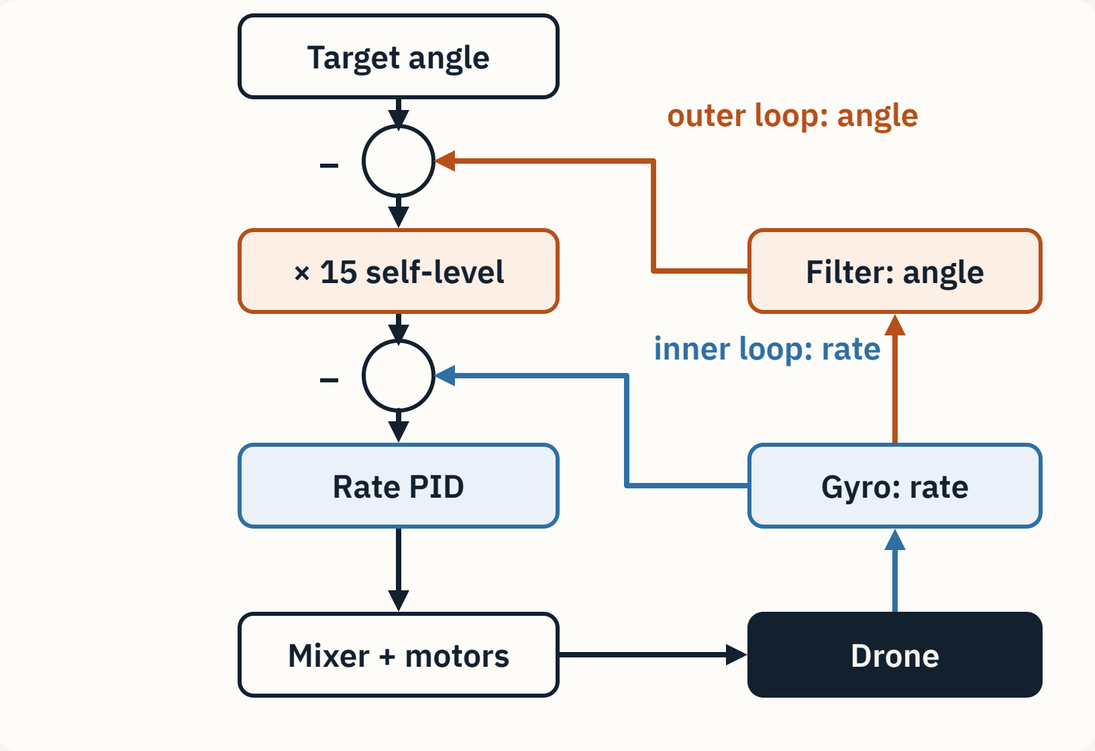
Outer loop: angle error × 15 → how fast to rotate back (a rate target).

Inner loop: rate target – gyro rate → the full PID → the mixer.

Why two loops? The fast rate loop fights quick bumps; the angle loop only decides how fast to come back.

EXAMPLE

Angle 4°, target 0° → outer asks –60 °/s → gyro reads –20 °/s → inner error +40 → push harder back.



PID CONTROL

Self-level in code

```
float roll_err_deg = angle_roll_deg -
sp_roll_deg;
roll_level_adjust = constrain(
    roll_err_deg * LEVEL_SENSITIVITY, -200.0f,
200.0f);
// The level term is SUBTRACTED.
roll_setpoint = sp_roll_rate - roll_level_adjust;
```

Angle error → times 15 → a rate in °/s, clamped to ±200.

Subtracted: tilted too far one way, ask to rotate the other way.

sp_roll_rate adds the pilot's own rate command (SBUS) on top.

EXAMPLE

Tilted +5° past the target → adjust = 5 × 15 = 75 → rate setpoint −75 °/s → the drone rotates back.

PID CONTROL

A real bug: the self-level sign

One character, + or −, decided whether the drone could fly.

Before (bug)

roll_setpoint = sp_roll_rate + roll_level_adjust

Tilted +5° → asks for +75 °/s: MORE tilt

Positive feedback: the drone flipped

After (fix)

roll_setpoint = sp_roll_rate − roll_level_adjust

Tilted +5° → asks for −75 °/s: back to level

test/level_test.cpp simulates both

level_test keeps the old formula as a negative control, so the test cannot pass by accident.

PID CONTROL

LEVEL_SENSITIVITY = 15

15

°/s of return rate per degree of angle error

±200

°/s cap on the self-level rate

≈ 13°

error at which the cap is reached (200 ÷ 15)

Example: 2° error → 30 °/s back. 20° error → 300 → capped at 200 °/s.

PID CONTROL

From the sticks to setpoints

```
// SBUS: rate mode, 16 µs dead-band around centre
if (rc[CH_ROLL] > 1508)
  sp_roll_rate = (rc[CH_ROLL] - 1508) /
SETPOINT_DIVISOR;
else if (rc[CH_ROLL] < 1492)
  sp_roll_rate = (rc[CH_ROLL] - 1492) /
SETPOINT_DIVISOR;
// Phone MANUAL: angles, capped at ±25°
sp_roll_deg = constrain(cmd_roll_deg,
                        -MAX_TILT_DEG,
MAX_TILT_DEG);
```

Dead-band 1492–1508: a stick resting near centre commands nothing.

÷ 3 (SETPOINT_DIVISOR): full stick 2000 → (2000 – 1508) ÷ 3 = 164 °/s.

The phone sends tilt **angles**; SBUS sends **rates**.

Default gains and how to tune them

Axis	Kp	Ki	Kd	Phone range (P / I / D)
Roll	1.3	0.04	18	0–100 / 0–20 / 0–200
Pitch	1.3	0.04	18	0–100 / 0–20 / 0–200
Yaw	4.0	0.02	0	0–100 / 0–20 / 0–200

Tuning order: I = 0 → raise P until it shakes, back off 30% → add D to stop bounce → add I to stop drift → save.

Check your understanding

Why does a drone need I if P already corrects?

Hint: what does P do when the error is small but constant?

Why is I reset when the throttle is at idle?

Hint: what grows while the drone sits on grass?

Why does too much D make the motors hot?

Hint: what does D do to sensor noise?

Why is the self-level term subtracted?

Hint: tilted +5°, which way must it rotate?

07

Mixer and motors

Three corrections and a throttle become four motor commands.

MIXER AND MOTORS

The mixer: four lines

```
motor_rf = t - pitch + roll - yaw *
YAW_DIRECTION;
motor_rr = t + pitch + roll + yaw *
YAW_DIRECTION;
motor_lr = t + pitch - roll - yaw *
YAW_DIRECTION;
motor_lf = t - pitch - roll + yaw *
YAW_DIRECTION;
```

t = throttle, the same on all four.

Roll: + on the right pair, – on the left pair.

Pitch: + on the rear pair, – on the front pair.

Yaw: + on one diagonal (RR, LF), – on the other (RF, LR).

EXAMPLE

Each column of signs sums to zero: roll + + – – = 0, pitch – + + – = 0, yaw – + – + = 0.

Worked example: one loop of mixing

Worked example: one loop of mixing.

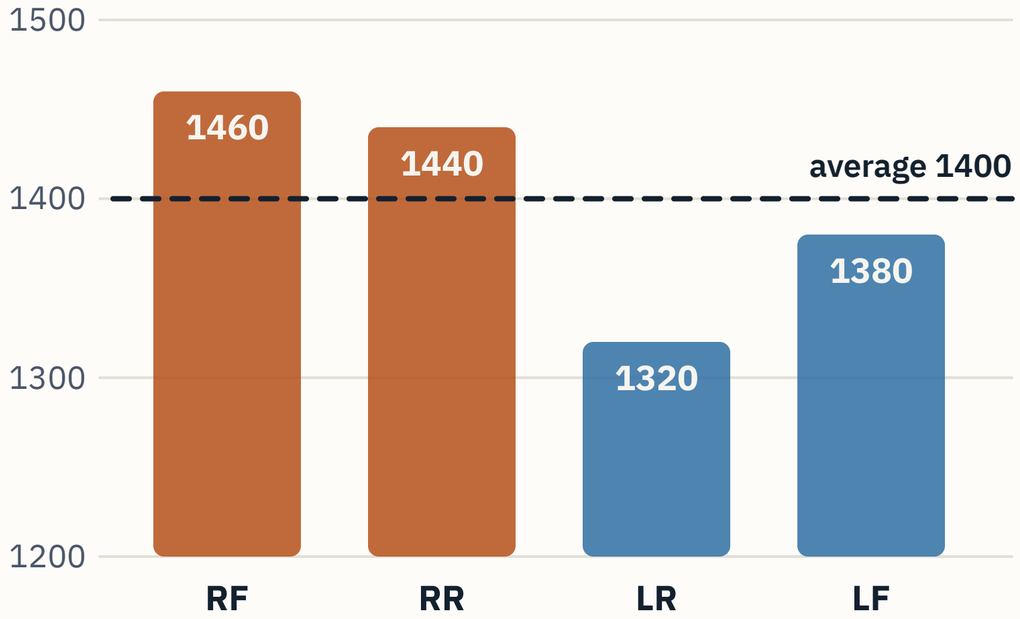
PID outputs this loop: **pitch −20, roll +50, yaw +10**. Throttle 1400.

RF = 1400 + 20 + 50 − 10 = **1460**; RR = 1400 − 20 + 50 + 10 = **1440**.

LR = 1400 − 20 − 50 − 10 = **1320**; LF = 1400 + 20 − 50 + 10 = **1380**.

EXAMPLE

Average = (1460 + 1440 + 1320 + 1380) ÷ 4 = 1400 — mixing moves thrust around; it never adds any.



MIXER AND MOTORS

Limits inside the mixer

```
int16_t t = constrain(throttle, RC_MIN, THROTTLE_MAX);
if (t > THROTTLE_ARM_MAX) {          // above 1050
  ... four mixer lines ...
  motor_rf = constrain(motor_rf, 1100, 2000);
  ...
} else {
  motor_rf = motor_rr = motor_lr = motor_lf = 1000;
  pidReset();   // avoid I-term windup at idle
}
```

Throttle is capped at **1800** to leave room for corrections.

Each motor stays between **1100** and **2000** while flying.

At idle throttle (≤ 1050) the motors stop and the PID memory is cleared.

usToDuty(): from 1000–2000 to 0–1023

```
inline uint16_t usToDuty(int16_t us) {
  if (us ⩽ 1000) return 0;
  if (us ⩾ 2000) return MOTOR_PWM_MAX;   // 1023
  return (uint16_t)(((uint32_t)(us - 1000)
                     * MOTOR_PWM_MAX) / 1000);
}
```

The code thinks in “µs” like a big-drone ESC, then converts at the last step.

Why keep 1000–2000? Familiar numbers, and the same mixer works if you ever swap to ESCs.

The uint32_t cast stops the multiply from overflowing 16 bits.

EXAMPLE

1500 → (500 × 1023) ÷ 1000 = 511 → about 50% duty. 1100 → 102 → 10% duty, the armed minimum.

Setting up the PWM hardware (LEDC)

```
for (uint8_t i = 0; i < 4; i++) {
  pinMode(pins[i], OUTPUT);
  digitalWrite(pins[i], LOW);
#if ESP_ARDUINO_VERSION_MAJOR ≥ 3
  ledcAttach(pins[i], MOTOR_PWM_FREQ_HZ, MOTOR_PWM_BITS);
#else
  ledcSetup(i, MOTOR_PWM_FREQ_HZ, MOTOR_PWM_BITS);
  ledcAttachPin(pins[i], i);
#endif
}
```

LEDC is the ESP32's PWM peripheral. It was made for LEDs, but drives MOSFETs just as well.

Two APIs: Arduino-ESP32 core 3.x changed the function names, so both are supported.

Pins are driven LOW first: motors off.

MIXER AND MOTORS

Why 20 kHz and 10 bits fit together

80 MHz

LEDC clock on the ESP32

4000

clock ticks per 20 kHz cycle (80 M ÷ 20 k)

1024

duty steps at 10 bits — fits inside 4000; 12 bits (4096) would not

Resolution × frequency is a trade-off. Ask why: what would you pick for a silent gimbal motor?

MIXER AND MOTORS

Everywhere the motors are stopped

Where	Why
setup()	safe state before anything else starts
Failsafe in loop()	no live input for 500 ms
flightEmergency()	CUT MOTORS from the app — always honoured
End of AUTO landing	ramp finished → disarm
DISARM, thrust-down, stick disarm	the pilot asked

MIXER AND MOTORS

Saturation: when a motor hits its limit

If one motor needs more than 2000 or less than 1100, it is clipped. The balance between the four is lost.

What you see

At full throttle the drone feels less stable; it may yaw or wobble.

Why

The clipped motor cannot give its share of the correction.

Built-in help

THROTTLE_MAX 1800 and PID_LIMIT ±400 make clipping less likely.

EXAMPLE

Throttle 1800, roll +300 → right motors 2100 → clipped to 2000. 100 units of roll correction are lost.

Check your understanding

Why do the mixer signs in each column add up to zero?

Hint: what happens to total thrust?

Why keep 1000–2000 if the motors use 0–1023 duty?

Hint: think about ESCs and readability.

Why is the throttle capped at 1800?

Hint: what does a correction need at full throttle?

Why 10-bit PWM and not 12-bit at 20 kHz?

Hint: count clock ticks per cycle.

08

Radio inputs: SBUS and CRTP

How sticks and phone taps become numbers in the loop.

RADIO INPUTS

An SBUS frame: 25 bytes

An SBUS frame: 25 bytes, every 7–14 ms.

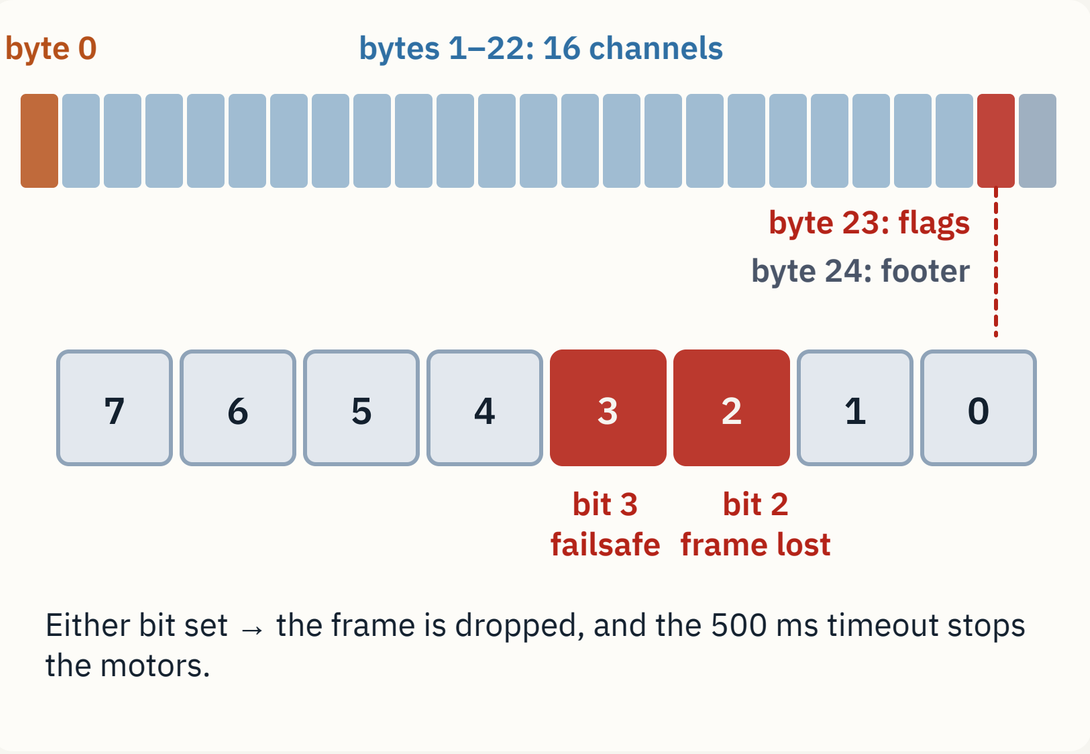
Byte 0: header 0x0F, the start of a frame.

Bytes 1–22: 16 channels × 11 bits, packed tight.

Byte 23: flags. **Byte 24**: footer (0x00, or 0x04/0x14/0x24/0x34 on SBUS2).

EXAMPLE

The old bug read the flags from byte 24 — the footer — so a receiver in failsafe was never noticed.



Either bit set → the frame is dropped, and the 500 ms timeout stops the motors.

RADIO INPUTS

Unpacking 11-bit channels

```
sbus_raw[0] = (b[1]      | b[2] << 8)
& 0x7FF;
sbus_raw[1] = (b[2] >> 3 | b[3] << 5)
& 0x7FF;
sbus_raw[2] = (b[3] >> 6 | b[4] << 2 | b[5] <<
10) & 0x7FF;
...    // 13 more lines, same pattern
```

Channels do not line up with bytes: each is 11 bits, so they straddle byte edges.

Shift moves the bits into place; **OR** joins the pieces; **& 0x7FF** keeps 11 bits.

Values run 0–2047; a normal stick uses about 172–1811.

EXAMPLE

b[1] = 0xE0, b[2] = 0x03 → (0xE0 | 0x03 << 8) & 0x7FF = 0x3E0 = 992, the stick centre.

RADIO INPUTS

A real bug: reading the wrong byte

Off by one byte, the failsafe flag was never seen.

Before (bug)

Flags were read from byte 24, the footer

Footer is always 0x00 → flags always look clear

A receiver in failsafe kept the motors running on stale sticks

After (fix)

Flags read from byte 23

Lost or failsafe frames are dropped

The 500 ms timeout then stops the motors; sbus_test.cpp pins it

RADIO INPUTS

Finding frames and scaling the sticks

```
while (Serial2.available()) {
  uint8_t c = Serial2.read();
  if (sbus_idx == 0 && c != 0x0F) continue;  // wait for
header
  sbus_buf[sbus_idx++] = c;
  if (sbus_idx >= SBUS_FRAME_LEN) {
    sbus_idx = 0;
    if (sbusFrameUsable(sbus_buf)) sbusDecode(sbus_buf);
  }
}
// in sbusDecode: 172..1811 -> 1000..2000
rc[i] = constrain(map(sbus_raw[i], 172, 1811, 1000,
2000),
                  1000, 2000);
```

Non-blocking: reads only what has arrived, never waits.

map() rescales to the 1000–2000 range the rest of the code uses.

CH5 becomes a simple switch: above 1024 → 2000, else 1000.

RADIO INPUTS

Arming with an RC transmitter

Action	Throttle	Yaw stick or CH5	Hold
Arm	below 1050	yaw right (above 1700) or CH5 high	1 s
Disarm	below 1050	yaw left (below 1300)	1 s
Disarm (switch)	any	CH5 dropped low, once a switch has been seen	1 s

Every arming path also needs the battery at or above 3.3 V (armChecksOk).

RADIO INPUTS

CRTP: the phone's language

CRTP: the first byte says where the packet goes.

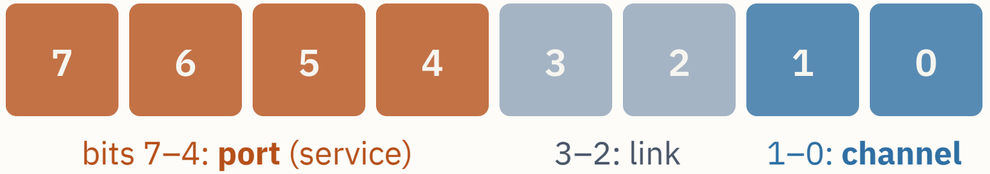
CRTP comes from the Crazyflie project; the ESP-Drone apps speak it.

The header's high 4 bits pick the **port**, the low 2 bits the **channel**.

crtpDispatch() reads the port and calls the right handler.

EXAMPLE

port = (byte >> 4) & 0x0F; channel = byte & 0x03. For 0x71: port 7, channel 1.



0x30	0011 0000	port 3, ch 0 → commander (sticks)
0x70	0111 0000	port 7, ch 0 → flight control
0x71	0111 0001	port 7, ch 1 → navigation
0x25	0010 0101	port 2, ch 1 → read a parameter

RADIO INPUTS

Framing and the checksum

```
uint8_t crtpChecksum(const uint8_t *data, size_t len) {
  uint32_t sum = 0;
  for (size_t i = 0; i < len; i++) sum += data[i];
  return (uint8_t)(sum & 0xFF);
}
// on receive (UDP / SPP):
if (crtpChecksum(data, len - 1) ≠ data[len - 1]) return;
crtpDispatch(data, len - 1);
crtp_last_rx_ms = millis();     // the link is alive
```

Over UDP and SPP, each packet ends with one checksum byte: the low 8 bits of the sum.

A bad checksum → the packet is dropped silently.

A good packet also refreshes **crtp_last_rx_ms**, used by the failsafe.

EXAMPLE

Packet 0x70 0x02 → sum 0x72 → checksum byte 0x72. Sent on the wire: 70 02 72.

RADIO INPUTS

Three ways to carry CRTP

Transport	Details	Framing
Wi-Fi UDP	drone 192.168.43.42, listens on 2390, replies to 2399	CRTP + checksum
Bluetooth SPP	serial over Bluetooth Classic (Android)	CRTP + checksum
Bluetooth LE	GATT writes and notifications (iPhone)	19-byte segments, no checksum

RADIO INPUTS

The commander packet (port 3)

Bytes	Field	Type	Meaning
0	header	0x30	port 3, channel 0
1 – 4	roll	float	tilt angle, degrees
5 – 8	pitch	float	tilt angle, degrees
9 – 12	yaw	float	yaw rate, °/s
13 – 14	thrust	uint16	0 – 65535

Sanity checks: NaN or infinity → ignored; |roll| or |pitch| over 180, or |yaw| over 2000 → ignored.

RADIO INPUTS

CRTP ports used here

Port	Name	Handled?	Used for
0	console	send only	text messages to the app (low battery, arm refused)
2	params	yes	read and write tunable values
3	commander	yes	stick packets in MANUAL
5	logging	yes	streaming telemetry, up to 3 blocks
7	generic	yes	ch 0 flight control, ch 1 navigation
4, 15	memory, link	ignored	not needed on this board

RADIO INPUTS

Parameters: the TOC and range checks

```
{ "pid", "roll_kp", PT_FLOAT, 4, TOC_ACCESS_RW,
  &pid_p_gain_roll, 0.0f, 100.0f },
{ "flight", "takeoff_ms", PT_UINT16, 2,
TOC_ACCESS_RW,
  &takeoff_ms, 100.0f, 20000.0f },
// on write:
if (!isfinite(v)) return;     // refused
v = paramClampF(e, v);        // clamp into [lo,
hi]
```

TOC = table of contents: the app downloads the list of names, types and access.

Each read-write entry carries a **lo** and **hi** range.

The reply echoes what was actually stored, so the app shows the truth.

EXAMPLE

The app writes roll_kp = 1000 → stored as 100 (the hi limit) → the echo tells the app 100.

RADIO INPUTS

Logging: streaming data to the phone

The app picks variables from the log TOC and asks for them at a period. The drone then sends them by itself.

Up to 3 blocks

Each with up to 12 variables and its own period (in 10 ms steps).

Groups

pm.vbat, stabilizer, controller, motor, flight, nav, pid (P/I/D per axis).

Packet

block id, 24-bit timestamp, then the values straight from the live variables.

EXAMPLE

Block 1, period 25 (× 10 ms = 250 ms): pm.vbat, flight.mode, flight.armed → 4 packets a second.

RADIO INPUTS

Port 7: the flight-control commands

Code	Command	What it does
0x01	SET_MODE	MANUAL, HOLD, AUTO, FOLLOW, RTH (refused while battery critical)
0x02 / 0x03	TAKEOFF / LAND	start the ramp up / the ramp down
0x04	SET_HOVER	hover thrust, clamped to 8000 – 55000
0x05	EMERGENCY	motors off now — always honoured
0x07 / 0x08	ARM / DISARM	ARM only from idle; DISARM always
0x09 – 0x0B	SAVE / DEFAULTS / LOAD	settings in flash; SAVE refused while armed
0x10	LINK	change transport profile and reboot; refused while armed

RADIO INPUTS · ASK WHY

Check your understanding

Why must the SBUS failsafe flag be checked at all?

Hint: what does a receiver send after losing the transmitter?

Why does the CRTP header put the port in the high bits?

Hint: how does crtpDispatch find the handler?

Why echo the stored value after a parameter write?

Hint: what if the drone clamped the value?

Why must every port 7 command be idempotent?

Hint: what does the app do every 500 ms?

09

Flight modes and safety

What the drone does on its own, and every layer that stops it doing harm.

MODES AND SAFETY

Five flight modes

Mode	Who flies	What the drone holds
MANUAL	pilot (phone sticks or RC)	nothing automatic — sticks command tilt or rate
HOLD	drone	level attitude, locked heading, a fixed thrust
AUTO	drone	take-off ramp → hover → landing ramp
FOLLOW	drone + phone GPS	tilts toward the phone's target offset
RTH	drone	tilts back toward the launch point

Modes apply to the phone path only. SBUS always flies MANUAL.

MODES AND SAFETY

AUTO is a small state machine

AUTO is a small state machine: one simple rule per state.

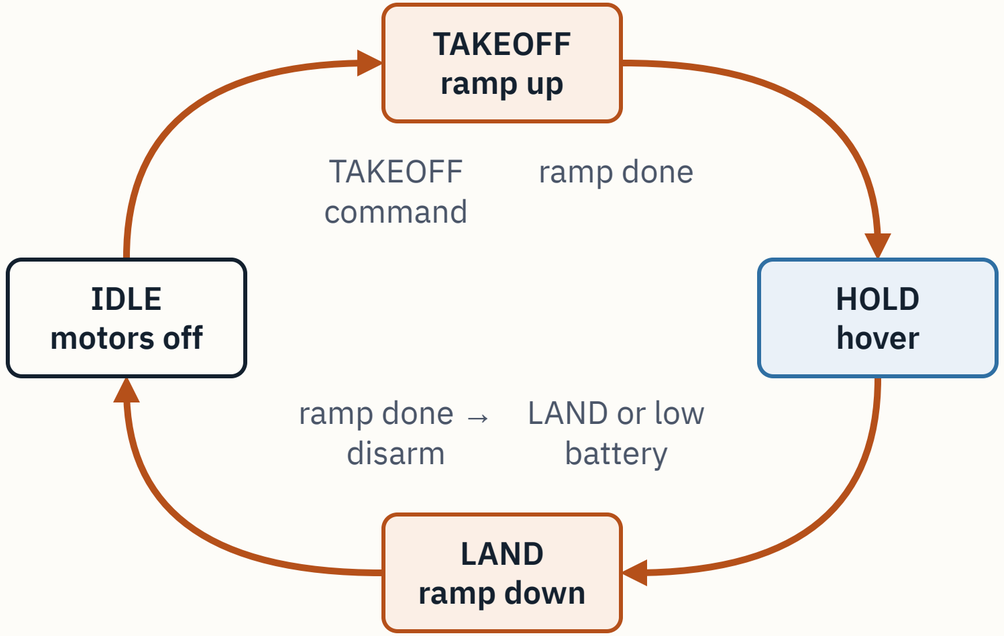
TAKEOFF moves to HOLD by itself when the ramp ends.

HOLD lasts until LAND is tapped or the battery goes critical.

LAND finishes by disarming and going back to MANUAL.

EXAMPLE

Tap Take off → 2 s climb → hover → tap Land → 2.5 s descent → disarmed.



MODES AND SAFETY

The take-off and landing ramps

Take-off and landing are straight-line ramps.

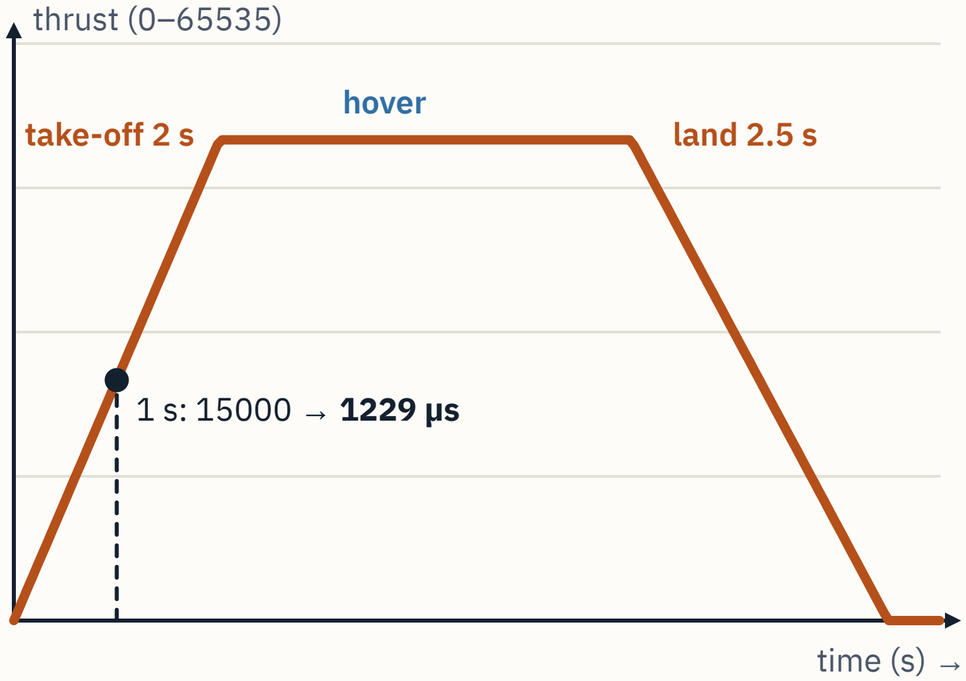
t runs from 0 to 1 over each phase, so thrust is a straight line.

Thrust (0–65535) becomes 1000–2000: **1000 + thrust ÷ 65535 × 1000**.

Landing is purely timed — there is no sensor that sees the ground.

EXAMPLE

hover_thrust 30000, takeoff_ms 2000 → at 1 s the thrust is 15000 → 1229 µs.



MODES AND SAFETY

Heading hold in HOLD and AUTO

Heading hold: always turn the short way.

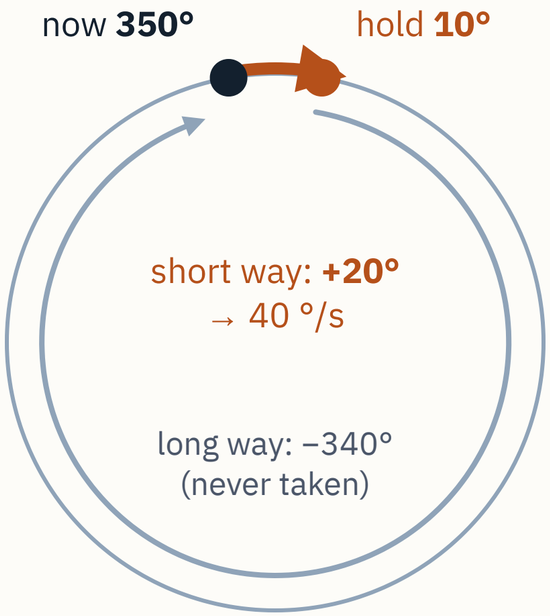
Heading error → yaw rate: **2 °/s per degree**, capped at 200 °/s.

The wrap (±360 when past ±180) picks the short way round.

The hold heading is captured on entering HOLD or at take-off.

EXAMPLE

Hold 10°, heading 350° → herr = −340 → +360 = +20 → yaw rate 40 °/s.



MODES AND SAFETY

Arming from the phone: three rules

Thrust low first

In MANUAL, thrust-up arms only after the app has shown thrust at zero. A phone that connects with the stick up cannot spin the props.

ARM only from idle

ARM is refused in HOLD, FOLLOW or RTH, where motors would jump to hover thrust. Use TAKEOFF there.

TAKEOFF arms safely

It checks the battery, sets the launch point and starts the ramp from zero.

One pre-arm check for every path

```
bool armChecksOk(const char *who) {
#if VBAT_ARM_CHECK
  if (vbat < VBAT_CELLS * VBAT_CELL_CRIT) {      // 3.3 V
    // rate-limited message, at most once a second
    Serial.printf("[ARM] %s refused: battery %.2f V\n",
who, vbat);
    crtpConsole("arm refused: low battery");
    return false;
  }
#endif
  return true;
}
```

Called by SBUS arming, CRTP thrust-up, ARM and TAKEOFF.

Why one function? A new check added here protects every path at once.

The message is rate-limited so a held stick cannot flood the console.

The failsafe: no input, no motors

The failsafe: no input, no motors.

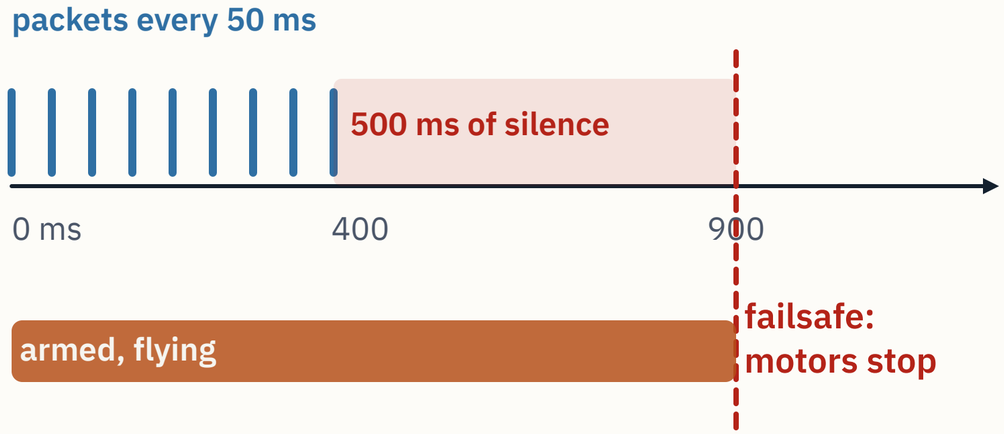
Each input is “alive” if it was heard in the last **500 ms**.

rx_ok = phone alive OR SBUS alive.

No live input while armed → disarm, motors off, back to MANUAL.

EXAMPLE

Phone app crashes at 400 ms → at 900 ms the props stop, in the same loop the check fails.



MANUAL needs stick packets. HOLD / AUTO / FOLLOW / RTH accept any CRTP traffic.

MODES AND SAFETY

A real bug: time measured backwards

One millisecond, read in the wrong order, could cut the motors.

Before (bug)

Ages compared as unsigned numbers

A packet handled this loop can be 1 ms newer than *now*

now – last = –1 → wraps to ~49 days → failsafe → motors cut

After (fix)

(int32_t)(now – last) < FAILSAFE_MS

–1 ms is simply “very fresh”

Signed comparison still works across the 49-day wrap

MODES AND SAFETY

Who is in control? Some scenarios

Situation	phone_ok	sbus_ok	Result
Phone flying MANUAL, sending sticks	yes	—	phone flies
Phone in AUTO hover, only log traffic	yes	—	phone flies (AUTO)
Phone app closed, RC receiver fitted	no	yes	RC flies, manual rate
Phone gone, no receiver	no	no	failsafe: motors stop
RC transmitter off, receiver in failsafe	no	no (flag)	failsafe: motors stop

Critical battery must last 3 seconds

Critical battery must last 3 seconds.

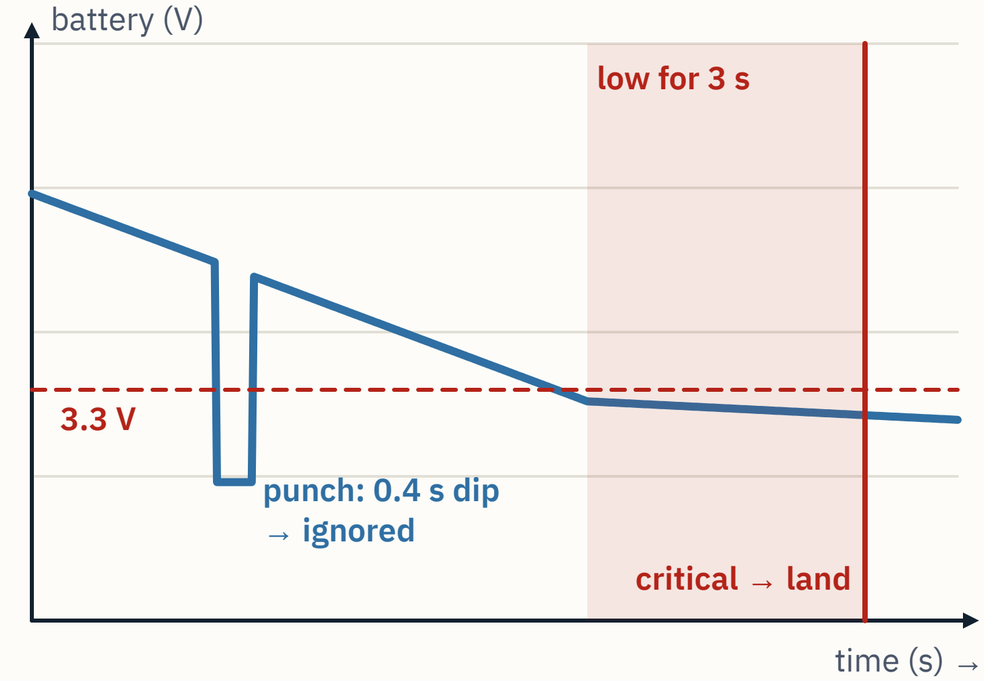
Two filters: the 0.2 s smoothing, then a **3 s hold**.

A throttle punch sags the voltage for a moment — that is not a dead battery.

The clock restarts as soon as the voltage recovers above 3.3 V.

EXAMPLE

Hovering at 3.28 V for 3 s → critical → in AUTO the drone lands itself; a manual pilot gets a warning.



MODES AND SAFETY

Critical battery → the drone lands itself

```
if (armed && vbat_crit) {
  if (use_crtp && flight_mode ≠ MODE_MANUAL) {
    if (!(flight_mode == MODE_AUTO && auto_state ==
AUTO_LAND))
      flightLand();                    // land once
  } else {
    // RC or MANUAL pilot: warn every 2 s, do not take
over
    crtpConsole("battery critical: land now");
  }
}
// and SET_MODE is refused while armed && vbat_crit
```

Automatic modes **land**; a human pilot gets a **warning**.

SET_MODE is refused so the app's regular mode resend cannot cancel the landing.

MODES AND SAFETY

Bad numbers never reach the motors

Every value from outside is checked before it is used.

Commander packets

NaN, infinity or absurd angles → ignored.

Parameter writes

NaN refused; others clamped into the TOC range.

Hover thrust

clamped to 8000 – 55000 from SET_HOVER, TAKEOFF and flash.

Flash at boot

a non-finite saved gain restores all defaults.

MODES AND SAFETY

Things that are refused while armed

SAVE

Flash erase stalls both CPU cores for milliseconds — a missed control loop.

LINK

Writes flash and reboots. A reboot in the air is a crash.

Serial link / sta / save

The bench console has the same rule: “refused while armed — disarm first”.

EXAMPLE

Tap Save in the Tune screen while hovering → reply status 1 → the app shows that the save was refused.

MODES AND SAFETY

CUT MOTORS, end to end

Tap
2 taps on Fly, 1 tap on Mission and Tune

→

App sends
port 7: 0x70 0x05 (EMERGENCY)

→

Firmware
flightEmergency(): disarm, motorStop, pidReset

→

Mode
back to MANUAL, AUTO idle

→

Reply
0x70 0x05 0x00 → app shows stopped

EMERGENCY has no conditions. It is the one command that always works.

EXAMPLE

Drone drifting toward a wall in HOLD → double-tap CUT MOTORS → props stop within one 4 ms loop of the packet arriving.

MODES AND SAFETY

Why a repeated TAKEOFF is ignored

```
// A retransmitted TAKEOFF must not restart the
ramp
if (armed && flight_mode == MODE_AUTO &&
    (auto_state == AUTO_TAKEOFF || auto_state ==
AUTO_HOLD))
  return true;                          // already
done
// Flying in another mode: never ramp from zero
bool spinning = motor_rf > 1000 || motor_rr >
1000 || ...;
if (armed && spinning) return false;
```

The app resends TAKEOFF every 500 ms until it sees the ack.

If the ack was lost, a second TAKEOFF arrives while climbing.

Restarting the ramp would drop thrust to zero — and drop the drone.

MODES AND SAFETY

Safety in layers

No single check is perfect, so they are stacked.

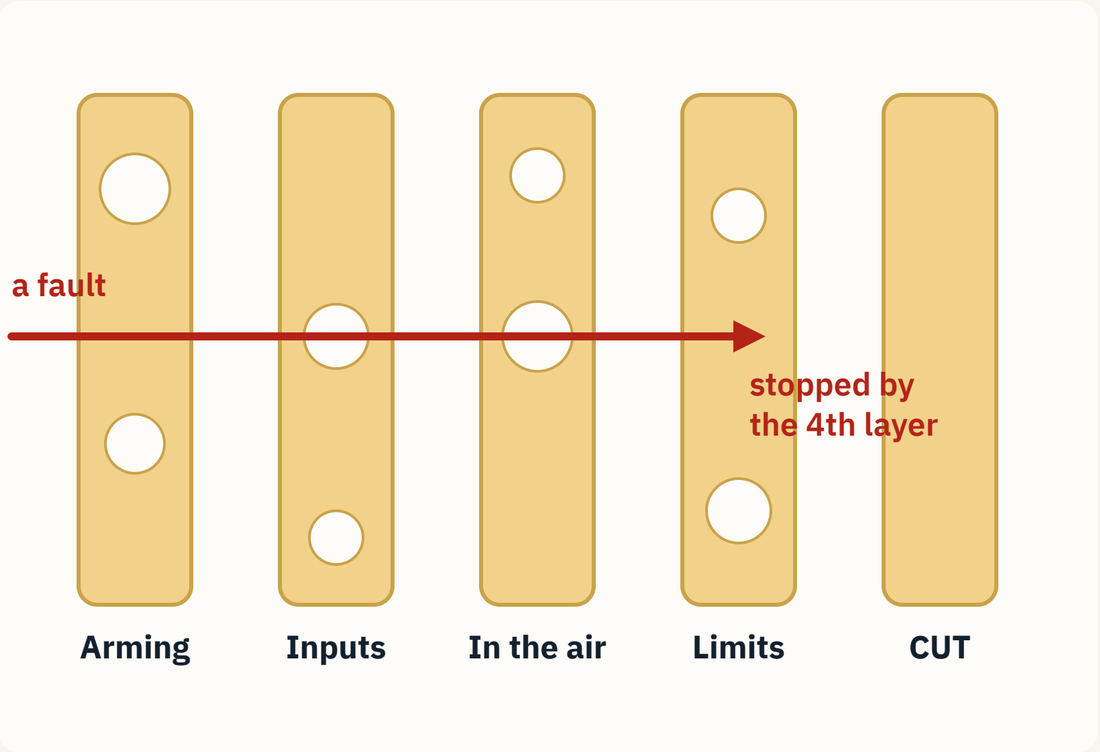
Arming: battery ≥ 3.3 V, thrust seen low, ARM only from idle.

Inputs: checksums, NaN and range checks, SBUS flags.

In the air: 500 ms failsafe, signed time, auto-land.

Limits: tilt ±25°, PID ±400, throttle 1800, motor floor 1100.

CUT MOTORS: honoured in every state.



Check your understanding

Why does MANUAL need stick packets but AUTO accepts any traffic?

Hint: what is the setpoint in each mode?

Why compare time ages as signed numbers?

Hint: what is 0 – 1 as an unsigned 32-bit number?

Why land automatically in AUTO but only warn in MANUAL?

Hint: who is flying in each case?

Why is EMERGENCY the only command with no conditions?

Hint: when would you ever want it refused?

10

Navigation

Flying toward the phone with no GPS on board — and being honest about the drift.

NAVIGATION

Dead reckoning: guessing position from motion

The phone knows where it is (GPS). The drone only knows how it has been accelerating.

Phone

Works out the target as metres north and east of the launch point, and sends it.

Drone

Adds up its own acceleration to guess how far it has moved since launch.

Controller

Tilts toward the difference between the target and the guess.

EXAMPLE

Walking across a dark room by counting steps: fine for a few metres, badly off after a long walk.

NAVIGATION

Acceleration → velocity → position

Acceleration → velocity → position, and the drift that comes with it.

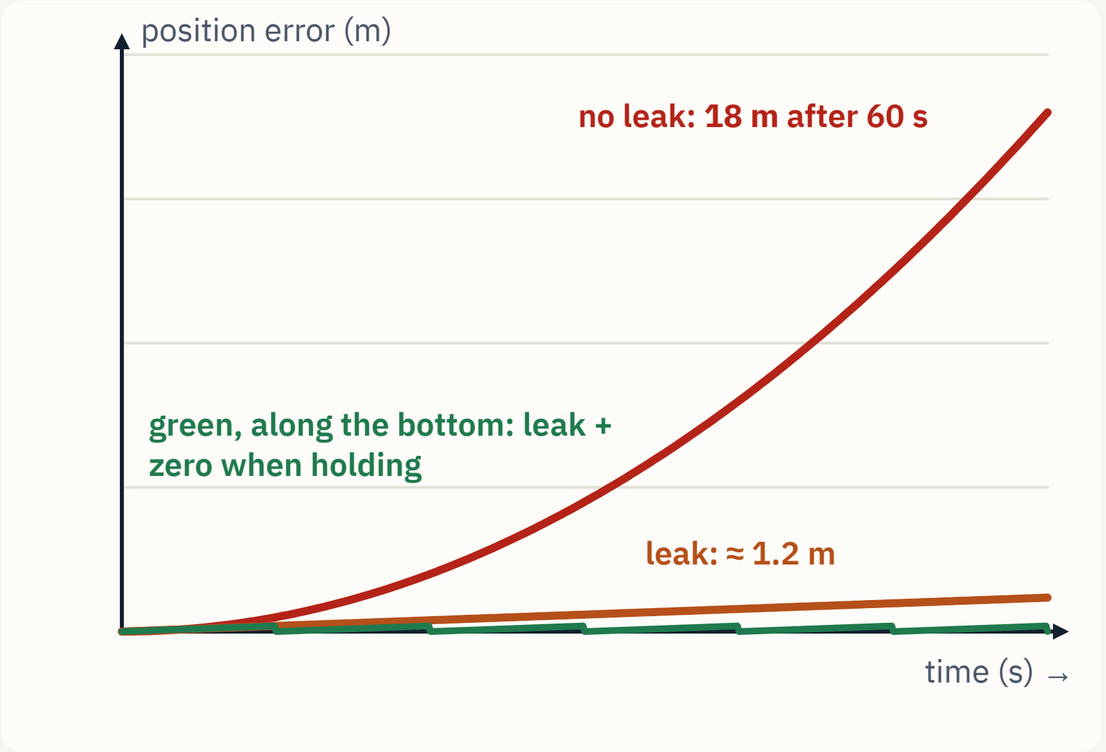
Accelerometer → remove gravity → rotate → integrate to velocity → integrate to position.

Every integration adds up small errors: a tiny bias grows with **time squared**.

Three fixes: a velocity leak, zero velocity when holding, reset at launch.

EXAMPLE

Bias 0.01 m/s² → ½ × 0.01 × 60² = 18 m after a minute without the leak. (Holding interval is an illustration.)



NAVIGATION

Removing gravity

```
const float ax_b = raw_ax / NAV_ACC_LSB_PER_G *
G;   // m/s²
const float ay_b = raw_ay / NAV_ACC_LSB_PER_G *
G;
// gravity seen by the tilted sensor
const float gx_g = -G * sin(roll) * cos(pitch);
const float gy_g = +G * sin(pitch);
float fwd  = (ay_b - gy_g) * NAV_FWD_SIGN;
float left = (ax_b - gx_g) * NAV_LEFT_SIGN;
```

A tilted drone feels part of gravity on its flat axes, even when it is not moving.

The tilt angles from section 5 say how much; subtract it.

What is left is real horizontal acceleration.

EXAMPLE

Tilted 5° forward, not moving: ay_b ≈ 9.81 × sin 5° = 0.85 m/s². Subtract gy_g = 0.85 → fwd = 0.

NAVIGATION

A real bug: the accelerometer scale

Two numbers, 1,800 lines apart, had to agree. They did not.

Before (bug)

NAV_ACC_LSB_PER_G = 16384 (the ±2 g scale)

Sensor actually set to ±8 g (4096 per g)

Every acceleration read 4× too small → position 4× too small

After (fix)

NAV_ACC_LSB_PER_G = 4096

Comment: must match MPU_REG_ACCEL_CFG in mpuInit()

run.sh reads both values and fails if they disagree

NAVIGATION

The leaky integrator

The leaky integrator forgets old velocity.

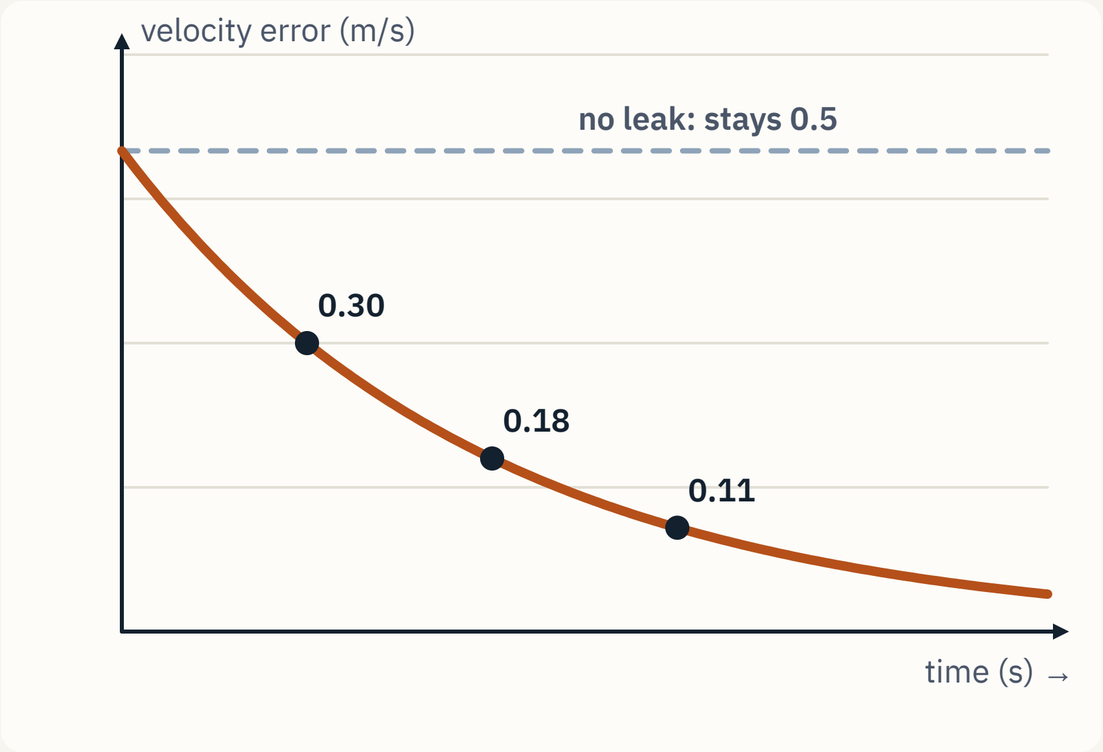
Each second, velocity keeps only **60%** of itself: a bias cannot build up forever.

powf(0.60, dt) makes that true per second whatever the loop rate (≈ 0.998 per loop).

When holding, velocity is set to zero — the one thing the drone knows for certain.

EXAMPLE

Cost: a long steady move is under-estimated a little — the right trade for a craft that mostly hovers.



NAVIGATION

The launch frame and NAV_CMD_ALIGN

The launch frame: +x where the nose pointed, +y to its left.

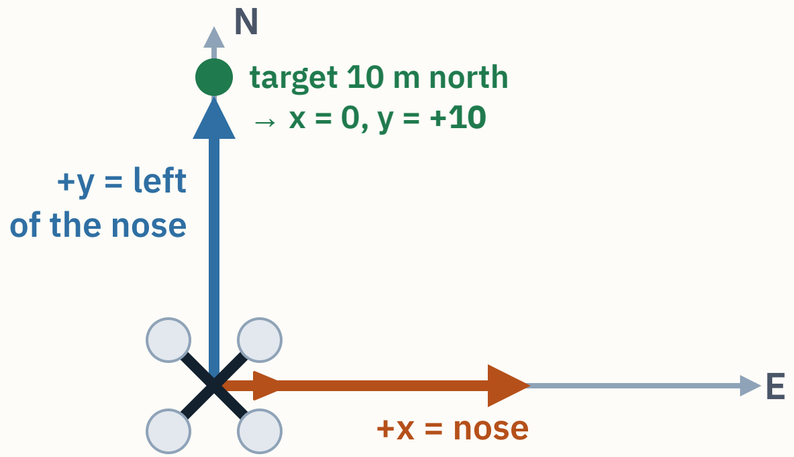
No compass on the drone: the phone sends the nose's bearing at launch — **ALIGN**.

Targets arrive as metres **north** and **east**; two lines turn them into the launch frame:

x = n·cos a + e·sin a, **y = n·sin a − e·cos a**.

EXAMPLE

a = 90°, target 10 m north: x = 10·0 + 0 = 0, y = 10·1 − 0 = 10 → 10 m to the left. ✓



Nose faces east (ALIGN says 90°), so north is to the drone's left.

NAVIGATION

Turning with the drone: two rotations

```
const float yaw_rel = (heading_deg -
heading_ref_deg)
                      * NAV_HEADING_SIGN *
DEG_TO_RAD;
// body → launch (navUpdate)
ax_w = fwd * cy - left * sy;
ay_w = fwd * sy + left * cy;
// launch → body (navSetpoints)
body_fwd  =  ux * cy + uy * sy;
body_left = -ux * sy + uy * cy;
```

The drone may yaw after launch. Acceleration is measured in the body, but position lives in the launch frame.

One rotation goes body → launch, the other is its exact reverse.

NAV_HEADING_SIGN = −1 makes the gyro's heading direction match the left-positive frame.

NAVIGATION

navSetpoints(): tilt toward the target

```
float ex = tgt_x - pos_x, ey = tgt_y - pos_y;
float dist = sqrtf(ex*ex + ey*ey);
if (dist < NAV_STOP_BAND_M || !tgt_valid) return;   // 0.6 m
float mag = constrain(dist * nav_kp, 0.0f, NAV_MAX_TILT_DEG);
float ux = ex / dist * mag;      // toward the target
float uy = ey / dist * mag;
// rotate into the body frame, then:
sp_pitch_deg = constrain(body_fwd,  -MAX_TILT_DEG, MAX_TILT_DEG);
sp_roll_deg  = constrain(body_left, -MAX_TILT_DEG, MAX_TILT_DEG);
```

A simple P controller: 1.2° of tilt per metre (nav_kp), capped at **12°**. Inside **0.6 m**, or with no target: stay level. RTH = target (0, 0).

Example: target 5 m ahead → 5 × 1.2 = 6° nose-down. 15 m ahead → 18 → capped at 12°.

NAVIGATION · ASK WHY

Check your understanding

Why does position error grow faster than velocity error?

Hint: how many times is the bias integrated?

Why set velocity to zero when holding?

Hint: what does the drone know for certain then?

Why does the phone send ALIGN at launch?

Hint: which sensor is missing on the drone?

Why is navigation tilt capped lower than manual tilt?

Hint: how sure is the drone of its position?

11

Testing and workflow

Proving the tricky maths on a PC, then building, flashing and reading the drone.

TESTING

Five test programs that run on your PC

Test	What it proves
frame_test	phone targets land in the same frame the dead reckoning uses
yawrot_test	body ↔ launch rotations turn the right way, with the sketch's sign
logcfg_test	log-block set-up survives retransmissions
level_test	self-level pulls back toward the target and never flips
sbus_test	lost and failsafe frames are rejected; good ones are accepted

Run them all with: **sh Esp32_drone/test/run.sh** — about 100 “ok” lines when everything passes.

TESTING

run.sh guards: the tests cannot go stale

```
check() {
  if ! grep -qF "$1" "$sketch"; then
    echo "STALE TEST: '$1' is no longer in
Esp32_drone.ino"
    exit 1
  fi
}
check 'roll_setpoint  = sp_roll_rate  -
roll_level_adjust;'
check 'if (f[23] & 0x08) return false;'
check 'tgt_y = n * sa - e * ca;'
```

The tests copy formulas from the sketch, because the sketch cannot be compiled on a PC.

A copy could quietly go out of date. **check** fails loudly if the exact line has changed.

Extra guards: mode bounds, accelerometer scale, NAV_HEADING_SIGN.

TESTING

Inside level_test: a tiny simulation

```
static float simulate(float angle0, float sp_deg,
                      float sign, float seconds) {
  float angle = angle0;
  for (int i = 0; i < (int)(seconds / 0.004f); i++) {
    float err = angle - sp_deg;
    float adj = clampf(err * 15.0f, -200, 200);
    float gyro = 0.0f + sign * adj;  // perfect rate loop
    angle += gyro * 0.004f;
    if (fabsf(angle) > 180.0f) return angle;   // flipped
  }
  return angle;
}
```

Models one axis at 250 Hz with a perfect rate loop.

sign −1 = today's firmware → the angle shrinks to the target.

sign +1 = the old bug → the angle grows and flips.

EXAMPLE

Start at +10°, target 0°: after 1 s with sign −1 → about 0°. With sign +1 → past 180°, flipped.

TESTING

What sbus_test checks

Case	Flags byte 23	Footer	Expected
normal frame	0x00	0x00	accepted
digital ch17/18 bits set	0x03	0x00	accepted
SBUS2 footer	0x00	0x14	accepted
receiver failsafe	0x08	0x00	rejected
frame lost	0x04	0x00	rejected
misaligned footer / bad header	—	0x55 / —	rejected
old byte-24 check (negative control)	0x08	0x00	wrongly accepted

TESTING

Adding a test of your own

1. Pick one formula or rule you are about to change, and copy those lines into a new test .cpp.
2. Write cases with known answers, worked out by hand: a good case and a bad case.
3. Keep the old version as a **negative control** so the test can be seen to fail.
4. Add a **check '...'** line to run.sh for each copied line, and a build-and-run step.
5. Run **sh run.sh**, then change the sketch, then run it again.

TESTING

The serial console on the bench

Command	Does
status	link, mode, auto state, armed, hover thrust, heading, battery
save / load / defaults	PID gains + flight settings in flash
link n	switch transport profile 0–5, then reboot
sta ssid pass	store Wi-Fi station credentials

Telemetry, once a second: **disarm | VBAT:3.92V | RC r:1500 p:1500 t:1000 y:1500 | ang p:+0.4 r:-0.2 | mot RF:1000 …**

TESTING

Build and flash

1. Install the **esp32 by Espressif** board package in Arduino IDE 2.x (or arduino-cli).
2. Open **Esp32_drone/Esp32_drone.ino**. Board: **ESP32 Dev Module**.
3. Partition scheme: **Huge APP** — required, the radios need about 1.7 MB.
4. Upload, then open the serial monitor at **115200**.
5. Keep the drone **flat and still** during the gyro calibration message.

WRAP-UP

Your learning path: five exercises

1. **Read** loop() and write down which numbered step changes each global variable.
2. **Bench**: props off, arm, tilt the drone by hand and watch which motors speed up.
3. **Log**: chart pid.roll_p, roll_i and roll_d in the app while hovering. Which term does the work?
4. **Test**: break the self-level sign on purpose and watch level_test fail. Then fix it.
5. **Extend**: change D to act on the measurement, or add mixer scaling — with a test first.

WRAP-UP

Glossary

Word	Meaning	Word	Meaning
Setpoint	what we want	Bias	a sensor's error at rest
Error	measured – wanted	Wind-up	I grows while it cannot act
Rate	rotation speed, °/s	Failsafe	stop when input is lost
Attitude	tilt angles	Idempotent	safe to apply twice
Duty	fraction of time ON	Dead reckoning	position from motion alone
CRTP	the phone protocol	TOC	list of params / log vars

WRAP-UP · ASK WHY

Deep questions to keep asking

Why is every sign bug in this firmware a flip, not a wobble?
Hint: positive feedback grows without limit.

Why does each missing sensor create exactly one limit in a mode?
Hint: map barometer, compass, GPS, rangefinder to HOLD, heading, FOLLOW, landing.

Why are most safety checks at the edges of the code?
Hint: where does untrusted data come in?

Why keep the buggy version inside every test?
Hint: how do you know a test can fail?

WRAP-UP

Sources

Source	Used for
Esp32_drone/Esp32_drone.ino	every code slide, value and comment quoted
Esp32_drone/test/*.cpp, run.sh	the testing section and bug stories
docs/INSTALL.md, docs/USER_GUIDE.md	build steps, CUT MOTORS, safety rules
TDK InvenSense MPU-6500 datasheet + register map	sensor ranges and registers
Espressif ESP32 datasheet + Technical Reference (LEDC)	chip features, PWM clock
Bitcraze CRTP documentation	the phone protocol layout
Alpha & Omega AO3416 datasheet	the motor MOSFET